



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

EQUIPO G

FIGUEROA MORALES BRYAN JAVIER
ROMERO MENDEZ BRYAM STEVEN
VELEZ LOPEZ RICARDO ELIAS

Asignatura: Aplicaciones Web

Docente: ING. GUERRERO ULLOA GLEISTON CICERON

Curso: Software - 'A' - 5TO NIVEL

Tema: Práctica Experimental Unidad I

[Repositorio GitHub](#)

1. Arquitectura de Aplicaciones Web y Diseño Inclusivo

1.1. Taxonomía Formal de Aplicaciones Web

La categorización de las aplicaciones web no solo se basa en criterios tecnológicos, sino que también muestra decisiones arquitectónicas que afectan directamente al rendimiento, la experiencia del usuario, la facilidad de mantenimiento y los costos operativos. Una taxonomía rigurosa posibilita que los equipos de desarrollo seleccionen el patrón más apropiado, tomando en cuenta tanto las exigencias funcionales como las no funcionales de un proyecto específico [1]

La forma más básica de presencia digital son las aplicaciones web estáticas. En ellas, el servidor proporciona al usuario los mismos archivos de HTML, CSS y JavaScript en cada solicitud, sin procesamiento dinámico en la parte del servidor. Cuando se distribuyen a través de redes de entrega de contenido (CDN), su mayor beneficio es la rapidez con que se entregan. No obstante, su falta de capacidad para personalizar contenido según el usuario o la información en tiempo real las restringe a situaciones como portafolios, documentos técnicos y páginas web corporativas con fines informativos [1]

Las aplicaciones web dinámicas son capaces de superar esta restricción porque producen HTML en el servidor mientras se ejecuta, normalmente interactuando con bases de datos y aplicando lógica empresarial antes de remitir la respuesta al cliente. Este enfoque se caracteriza por tecnologías como Ruby on Rails, Django, PHP y Node.js junto con Express. Cada requerimiento genera un ciclo completo de cliente-servidor, lo que podría resultar en una latencia más alta; no obstante, este inconveniente puede atenuarse mediante el uso de estrategias apropiadas de caché [1].

Las aplicaciones de página única (SPA) suponen una transformación de paradigma: no vuelven a cargar documentos HTML enteros cada vez que se interactúa, sino que cargan un solo documento inicial y lo actualizan al instante utilizando llamadas asíncronas al servidor (AJAX) y manipulando el Document Object Model (DOM). Frameworks como Vue.js, Angular y React son los más populares. Si bien esta arquitectura facilita la experiencia del usuario al suprimir las recargas de página, también complica la administración del estado de la aplicación y penaliza el tiempo que tarda en cargarse inicialmente (TTI), pues el navegador tiene que bajar, examinar y ejecutar un paquete de JavaScript mucho más grande antes de mostrar contenido visible [2].

Las aplicaciones web progresivas (PWA) extienden el modelo SPA incorporando tecnologías adicionales: el manifiesto web (manifest.json), que posibilita la instalación en la pantalla principal del dispositivo; y los Service Workers, que son scripts de JavaScript que se ejecutan en un hilo distinto al del navegador e interceptan las peticiones de red para aplicar complejas estrategias de caché. Las PWA tienen la capacidad de operar en modo sin conexión, ya sea total o parcial, recibir notificaciones push y sincronizar información en segundo plano [2], [3].

Las arquitecturas monolíticas y las basadas en microservicios se encuentran en el polo opuesto del espectro. Una arquitectura monolítica combina todos los elementos —lógica de negocio, acceso a datos y presentación— en una sola unidad desplegable. Su mayor beneficio es la sencillez del desarrollo inicial; sin embargo, el entrelazado de componentes complica el escalado autónomo de módulos y ralentiza los ciclos de despliegue conforme el sistema crece. Los microservicios dividen la aplicación en pequeños servicios independientes, cada uno con su propia base de datos y comunicándose a través de APIs definidas [4]. Estudios empíricos demuestran que en sistemas con cargas moderadas, la arquitectura monolítica puede ser más eficiente en latencia; pero en

sistemas de alta concurrencia con escalado selectivo por componente, los microservicios brindan beneficios significativos en disponibilidad y rapidez de desarrollo [5].

1.2. Ciclo de Vida de una Solicitud HTTP/HTTPS

Desde que un usuario introduce una URL en el navegador hasta que el contenido se renderiza en pantalla, se lleva a cabo una serie de operaciones técnicas coordinadas que abarca varios protocolos y capas del modelo OSI [1]. Entender este ciclo es esencial para detectar problemas de rendimiento, aplicar medidas de seguridad eficaces y crear arquitecturas de frontend consistentes.

La etapa inicial es la resolución DNS (Domain Name System), un sistema distribuido que convierte nombres de dominio comprensibles para las personas en direcciones IP procesables por los dispositivos de red. Cuando el navegador recibe una URL, primero revisa su caché local; si no encuentra un registro válido, consulta al sistema operativo y, si tampoco resuelve localmente, contacta al servidor DNS configurado por el ISP. Este actúa como un resolver recursivo, consultando servidores raíz, TLD y finalmente el servidor autoritativo del dominio. La distancia geográfica y la carga de los servidores intermedios pueden hacer fluctuar el tiempo de resolución DNS entre 20 y 120 ms [1].

Después de obtener la dirección IP, el navegador establece una conexión TCP usando el three-way handshake (SYN → SYN-ACK → ACK). Para páginas HTTPS —estándar vigente para toda aplicación web moderna— se añade la negociación TLS. En TLS 1.3, actualmente preferida por los navegadores modernos, esta negociación requiere solo un round-trip (1-RTT) en conexiones nuevas y puede eliminarse mediante reanudación de sesión (0-RTT). Durante el handshake, cliente y servidor acuerdan algoritmos criptográficos, intercambian certificados y generan las claves de sesión [1].

El método HTTP define la semántica de la operación: GET obtiene recursos sin alterar el estado del servidor, es idempotente y cacheable; POST envía información para crear un recurso nuevo, no es idempotente; PUT sustituye un recurso existente de forma total y es idempotente; PATCH implementa cambios parciales y puede o no ser idempotente según la implementación; DELETE suprime el recurso identificado y es idempotente [6]. La elección correcta del método impacta directamente en cómo operan los caches intermedios (CDNs, proxies) y en la coherencia del estado de la aplicación.

Los códigos de estado HTTP se agrupan en familias semánticas: 2xx indica éxito (200 OK, 201 Created, 204 No Content); 3xx indica redirección (301 Moved Permanently, 304 Not Modified); 4xx indica errores imputables al cliente (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests); 5xx indica errores del servidor (500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable) [6]. Las cabeceras HTTP de seguridad merecen atención especial: HSTS instruye al navegador a usar exclusivamente HTTPS; CSP define las fuentes de contenido autorizadas mitigando XSS; X-Frame-Options previene ataques de clickjacking.

1.3. Principios WCAG 2.1 y Marco Legal en Ecuador

Las Pautas de Accesibilidad para el Contenido Web 2.1 (WCAG 2.1), publicadas por el W3C en junio de 2018, son el estándar internacional de referencia para garantizar y evaluar la accesibilidad del contenido digital. Se estructuran alrededor de cuatro principios esenciales cuyas iniciales forman el acrónimo POUR [7]:

- **Perceptible:** Los usuarios deben recibir toda la información a través de formas que sus sentidos disponibles puedan percibir. Los criterios AA incluyen el 1.4.3 (contraste mínimo 4.5:1 para texto normal y 3:1 para texto grande) y el 1.1.1 (texto alternativo para imágenes). Esto supone revisar sistemáticamente la paleta de colores y garantizar que los reproductores multimedia incluyan subtítulos [7].
- **Operable:** Los elementos de la interfaz deben poder operar con distintos dispositivos y modalidades de entrada. El criterio AA más crítico es el 2.1.1 (accesibilidad por teclado): toda funcionalidad debe poder ejecutarse usando únicamente el teclado. Esto tiene particular importancia en las SPAs, donde la gestión del foco después de transiciones de ruta o apertura de modales suele omitirse [7].
- **Comprensible:** La información y el manejo de la interfaz deben ser comprensibles para cualquier usuario. El criterio 3.3.1 establece que los errores de entrada del usuario deben identificarse y explicarse con texto; el 3.3.2 exige instrucciones o etiquetas cuando el contenido necesita entrada del usuario [7].
- **Robusto:** El contenido debe ser suficientemente robusto para ser interpretado por una amplia variedad de agentes de usuario, incluidas las tecnologías de asistencia. El criterio 4.1.2 exige que todos los componentes de la interfaz tengan nombre, rol y valor disponibles programáticamente [7].

En Ecuador, garantizar la accesibilidad web es además un deber legal. La Ley Orgánica de Discapacidades (LOD, 2012) establece que las personas con discapacidad tienen derecho a acceder a la información y comunicaciones en condiciones de igualdad. La Norma Técnica INEN 2620 adopta los criterios WCAG 2.0 para servicios digitales del sector público, exigiendo conformidad AA. El incumplimiento expone a las organizaciones a consecuencias legales y, más importante aún, a la exclusión sistemática de una parte significativa de la ciudadanía [7], [8].

Respecto a las herramientas de auditoría: WAVE (WebAIM) muestra visualmente las incidencias directamente sobre la página evaluada y alcanza el mayor ratio entre criterios WCAG cubiertos y errores detectados. Lighthouse (Chrome DevTools) realiza una evaluación más exhaustiva que abarca rendimiento, buenas prácticas y SEO, usando axe-core como motor de análisis de accesibilidad. Axe DevTools proporciona una cobertura más amplia de criterios WCAG automatizables y menor tasa de falsos positivos [9]. Ninguna herramienta automatizada es suficiente por sí sola: se requiere combinación de herramientas automatizadas, revisión manual experta y pruebas con usuarios reales [8].

1.4. Análisis Crítico: SPA, Accesibilidad y Progressive Enhancement

La arquitectura SPA presenta tensiones inherentes con los principios de accesibilidad. Primero, el enrutamiento del lado cliente no avisa automáticamente a los lectores de pantalla acerca del cambio de contexto. En una aplicación web tradicional, el lector de pantalla avisa la cabecera de la página y lleva el foco al comienzo del documento al cargarse uno nuevo; en las SPA, esta transición debe gestionarse explícitamente moviendo el foco a un encabezado importante o a un contenedor marcado con `aria-live='polite'` [8].

Segundo, cuando se cargan componentes de forma diferida (lazy loading), los estados de carga no anunciados por las tecnologías de asistencia desorientan al usuario. El modelo indicado es incluir un mensaje accesible — normalmente un spinner textual con `role='status'` — que avise cuando el contenido está cargando y cuando ha completado la carga [10].

El principio de mejora progresiva (progressive enhancement) ofrece una solución estructural: la funcionalidad esencial debe operar sin JavaScript, y este se usa solo para mejorar la experiencia en navegadores compatibles. En una PWA, esto implica feature detection antes de usar APIs potencialmente no disponibles: si `navigator.serviceWorker` está disponible, se registra el Service Worker; si se ha otorgado permiso, se usan las notificaciones push. Este enfoque garantiza que la aplicación funcione correctamente incluso en contextos con runtime limitado, mejorando la resiliencia general del sistema [2], [3].

2. HTML5 Semántico y Formularios Avanzados

2.1. Semántica Estructural en HTML5

La semántica de HTML5 no es un elemento decorativo del marcado, sino una herramienta esencial que permite que el contenido de una página web sea significativo para las tecnologías de asistencia y los motores de búsqueda. La estructura de datos que los navegadores presentan a los lectores de pantalla como JAWS y NVDA es el árbol de accesibilidad, derivado del DOM semántico [10]. Si el marcado carece de semántica adecuada, quien utiliza tecnología de asistencia no puede navegar por la estructura del contenido de manera eficaz.

La especificación WHATWG establece criterios precisos para la elección entre los elementos contenedores más comúnmente confundidos [6]. El elemento `<article>` se usa para contenido autónomo e independiente que tenga sentido por sí mismo fuera del contexto de la página —una entrada de blog, un artículo periodístico, un comentario de usuario o una tarjeta de producto—. El elemento `<section>` engloba contenido relacionado temáticamente que forma parte de un conjunto más grande y normalmente contiene un encabezado; su uso es necesario cuando el bloque debe aparecer en la tabla de contenidos del documento. El elemento `<div>` es un contenedor genérico sin semántica propia, cuyo propósito es agrupar elementos por razones de presentación o scripting cuando ningún otro elemento semántico resulta apropiado. El uso indiscriminado de `<div>` en lugar de elementos semánticos daña la accesibilidad y el SEO [1], [6].

La especificación establece además que `<main>` debe ser único por página y contener el contenido principal, diferenciado de la navegación, encabezados y pies de página [6]. Los usuarios de lectores de pantalla pueden saltar directamente al contenido principal mediante atajos de teclado gracias a su presencia. El elemento `<aside>` representa contenido relacionado con el contenido principal pero separable de él —barras laterales, bloques de citas destacadas, publicidad relacionada—. La jerarquía de encabezados (`<h1>` a `<h6>`) debe reflejar la estructura lógica del documento: se recomienda un único `<h1>` coincidente con el título principal del contenido, seguido de `<h2>` para secciones principales y `<h3>` para subsecciones, manteniendo jerarquía sin saltos. Un usuario de lector de pantalla utiliza los encabezados como mapa de navegación; una jerarquía incoherente destruye esta capacidad [7], [10].

2.2. WAI-ARIA en Profundidad

WAI-ARIA (Web Accessibility Initiative — Accessible Rich Internet Applications) es una especificación que ofrece un conjunto de atributos para mejorar la semántica de los elementos HTML en las tecnologías asistenciales, especialmente en interfaces interactivas y dinámicas. La versión 1.2 fue lanzada por el W3C en junio de 2023[10]. El modelo conceptual de ARIA se articula en tres categorías: roles, estados y propiedades.

La primera norma de uso de ARIA establece que se debe optar por el elemento o atributo HTML nativo que comunique la semántica deseada implícitamente antes de recurrir a ARIA [10]. Esta regla responde a evidencia empírica: según el informe WebAIM Million 2026, las páginas con atributos ARIA tienen en promedio 59.1 errores de accesibilidad detectables, mientras que las que no los contienen presentan solo 42. La directiva 'sin ARIA es mejor que ARIA mal aplicada' resume esta realidad [9], [10].

- **Roles fundamentales:** `role="navigation"` demarca regiones de navegación cuando no puede usarse `<nav>`; `role="dialog"` identifica un cuadro de diálogo modal e instruye al lector de pantalla a anunciar su apertura; `role="alert"` es una zona activa implícita con `aria-live="assertive"` para mensajes urgentes

que necesitan atención inmediata; `role="status"` con `aria-live="polite"` es apropiado para avisos no urgentes como actualizaciones de progreso o confirmaciones de guardado [10].

- **Estados ARIA:** `aria-expanded="true/false"` comunica si un elemento colapsable está abierto o cerrado; `aria-selected` se aplica a elementos de listas de selección, pestañas o cuadrículas; `aria-checked` comunica el estado de checkboxes o toggles personalizados. Todos deben actualizarse programáticamente cuando el estado del componente cambie [10].
- **Propiedades ARIA:** `aria-label` asigna un nombre accesible directamente al elemento cuando no hay texto visible cercano como etiqueta; `aria-labelledby` referencia el ID de un elemento existente cuyo texto servirá como nombre accesible; `aria-describedby` proporciona una descripción complementaria con instrucciones adicionales; `aria-live` controla cómo se anuncian los cambios dinámicos ("`polite`" espera inactividad del usuario; "`assertive`" interrumpe inmediatamente) [7], [10].

2.3. Formularios HTML5 y Experiencia de Usuario Accesible

Los formularios HTML5 constituyen el principal mecanismo de entrada de datos del usuario en las aplicaciones web. La especificación HTML5 estandarizó 22 tipos de elementos `<input>`, muchos de los cuales tienen implicaciones directas en la experiencia de usuario en dispositivos móviles [6]. El tipo `type="tel"` activa el teclado numérico en Android e iOS; `type="email"` habilita teclados con acceso directo a '@'; `type="date"` y `type="datetime-local"` proveen selectores nativos del sistema operativo optimizados para táctil; `type="range"` genera un control deslizante; `type="color"` incluye un selector de color nativo [6].

La validación nativa de HTML5 se activa a través de atributos como `required`, `pattern`, `min`, `max`, `minlength` y `maxlength`. Sus ventajas incluyen: no requiere JavaScript, está integrada con tecnologías de asistencia mediante los roles implícitos de los controles de formulario, y previene el envío al servidor de datos inválidos [6]. Sus limitaciones son: los mensajes de error nativos no son personalizables fácilmente vía CSS; su aspecto difiere entre plataformas; y para validación en tiempo real (`inline validation`) mientras el usuario escribe, debe complementarse con JavaScript [7], [10].

El patrón de validación personalizada accesible con JavaScript debe interceptar el evento `"input"` o `"blur"` del campo para validar en tiempo real, y el evento `"submit"` del formulario para comprobación global. Al identificar un error, se marca el campo con `aria-invalid="true"` y se añade un mensaje al DOM vinculado al campo mediante `aria-describedby`. El contenedor del mensaje puede marcarse con `role="alert"` para anunciarlo inmediatamente [10]:

```
<!-- Marcado HTML base -->
<div class="campo">
  <label for="email">Correo electrónico</label>
  <input type="email" id="email" name="email"
        aria-describedby="email-error" required>
  <span id="email-error" role="alert" hidden></span>
</div>
// Validación accesible con JavaScript
const campo = document.getElementById('email');
const error = document.getElementById('email-error');
```

```

campo.addEventListener('blur', () => {
  if (!campo.validity.valid) {
    campo.setAttribute('aria-invalid', 'true');
    error.textContent = 'Introduce un correo válido.';
    error.hidden = false;
  } else {
    campo.removeAttribute('aria-invalid');
    error.hidden = true;
  }
});

```

2.4. APIs HTML5 para el PFC: Web Storage

La selección del mecanismo de almacenamiento del lado cliente depende de criterios de persistencia, capacidad y seguridad. `localStorage` persiste los datos entre sesiones del navegador hasta que el usuario los elimina explícitamente; tiene una capacidad de aproximadamente 5 MB por origen y es accesible desde cualquier contexto JavaScript de la misma origen. `sessionStorage` mantiene los datos únicamente durante la sesión activa del navegador —se elimina al cerrar la pestaña—, con la misma capacidad de ~5 MB. Las cookies, con capacidad de apenas 4 KB, se transmiten automáticamente con cada petición HTTP al servidor, lo que las hace apropiadas para tokens de sesión, pero las expone a riesgos de seguridad adicionales si no se configuran con los atributos `HttpOnly` y `Secure` [6].

El riesgo de seguridad más relevante de `localStorage` y `sessionStorage` es la vulnerabilidad a ataques XSS: si un atacante logra inyectar código JavaScript malicioso, puede leer todo el contenido del almacenamiento. Por esta razón, jamás debe almacenarse información sensible como contraseñas o tokens de autenticación en estos mecanismos sin cifrado adicional.

3. CSS3 Avanzado: Grid, Flexbox y Responsividad

3.1. CSS Grid vs. Flexbox — Análisis Técnico Comparativo

El diseño de interfaces modernas se fundamenta en la coexistencia de dos especificaciones del W3C: CSS Grid Layout y CSS Flexible Box Layout (Flexbox). Ambos modelos resuelven el problema del posicionamiento dinámico, pero operan bajo paradigmas estructurales ortogonales dentro de la arquitectura de las hojas de estilo [11].

- CSS Grid se define formalmente como un sistema de diseño bidimensional. Su arquitectura permite estructurar interfaces dividiendo el espacio en filas y columnas simultáneamente [12]. El contenedor establece una cuadrícula rígida o elástica mediante propiedades como `grid-template-columns` y unidades fraccionales (`fr`), permitiendo delimitar áreas explícitas mediante `grid-template-areas` y controlar la adaptabilidad mediante funciones como `minmax()`. El espacio se predefine antes de posicionar el contenido.
- Flexbox se define formalmente como un sistema de diseño unidimensional. Su propósito es la distribución de elementos a lo largo de un único eje lineal —horizontal (`row`) o vertical (`column`)—. Flexbox calcula el espacio disponible basándose en las dimensiones intrínsecas de sus elementos hijos y los distribuye de manera fluida, siendo ideal para flujos continuos donde el tamaño del contenedor se adapta dinámicamente.

Criterio de decisión fundamental: usar Grid cuando el layout importa más que el contenido; usar Flexbox cuando el contenido importa más que el layout. A continuación, se presenta una comparativa analítica basada en ocho criterios técnicos fundamentales para el rendimiento y la maquetación de interfaces [13]:

Criterio Técnico	CSS Grid Layout	CSS Flexible Box (Flexbox)
1. Control del eje	Bidimensional: gestiona filas y columnas simultáneamente.	Unidimensional: un solo eje principal a la vez; el eje secundario es dependiente.
2. Alineación de celdas	Permite alineación independiente de celdas individuales dentro de sus coordenadas de red.	La alineación se aplica de forma colectiva sobre el eje (con excepciones vía <code>align-self</code>).
3. Soporte de gap	Soporte nativo histórico e integral para <code>row-gap</code> y <code>column-gap</code> sin colapso de márgenes.	Implementado de forma posterior en la especificación; soporte actual unificado en navegadores modernos.
4. Compatibilidad	Excelente en navegadores modernos; requiere prefijos en motores muy antiguos.	Máxima compatibilidad retrospectiva; comportamiento maduro en casi totalidad del software cliente.
5. Intrinsic sizing	Riguroso. Utiliza <code>min-content</code> y <code>max-content</code> calculando la rigidez estructural global.	Flexible y prioritario. El dimensionamiento intrínseco del contenido dicta la expansión del contenedor.
6. Orden visual vs. DOM	Permite disociación total entre orden del DOM y visualización espacial mediante coordenadas o áreas de rejilla.	Permite reordenación lineal mediante la propiedad <code>order</code> y direcciones inversas (<code>row-reverse</code>).
7. Dependencia estructural	Declarativo: El contenedor define el esqueleto y los elementos se inyectan en él.	Reactivo: El contenido define el tamaño del componente y empuja las fronteras del layout.
8. Complejidad del marcado	Reduce la necesidad de wrappers anidados en el HTML, manteniendo un DOM limpio.	Suele requerir múltiples capas de contenedores anidados para lograr estructuras visuales complejas.

3.2. Sistema de Diseño Responsivo Mobile-First

La adopción de una metodología Mobile-First constituye un estándar de optimización arquitectónica de CSS. El CSS base —diseñado para pantallas pequeñas— es conceptualmente el más restrictivo y simplificado, careciendo de layouts complejos de múltiples columnas. Las media queries se limitan exclusivamente a añadir y enriquecer progresivamente la interfaz a medida que el ancho de pantalla incrementa, minimizando overrides de selectores y reduciendo el peso del archivo final [14].

Breakpoints estándar de la industria en 2025:

- Dispositivos móviles (pantallas pequeñas): Base CSS sin media query o hasta 480px.
- Tabletas (orientación retrato): @media (min-width: 768px)
- Tabletas (orientación paisaje) / Laptops pequeñas: @media (min-width: 1024px)
- Pantallas de escritorio estándar: @media (min-width: 1200px)
- Pantallas de escritorio ultra-anchas: @media (min-width: 1440px)

Para tipografía fluida sin media queries excesivas, se implementa la función clamp():

```
font-size: clamp(1rem, 2.5vw, 1.5rem);
```

Esta directiva mantiene un tamaño base de 1rem en pantallas reducidas, escala dinámicamente al 2.5% del ancho del viewport, y se detiene al alcanzar un techo de 1.5rem, garantizando consistencia visual sin rupturas estéticas.

3.3. Variables CSS y Sistemas de Temas

Las propiedades personalizadas de CSS (Custom Properties) operan bajo la cascada de herencia nativa del documento web. Al ser declaradas en el selector de mayor jerarquía (:root), se vuelven accesibles para la totalidad de los nodos del árbol del DOM. No obstante, al seguir las reglas de especificidad, pueden ser redefinidas en ámbitos o componentes específicos para alterar localmente su comportamiento, aislando el impacto de los cambios [1].

Un sistema de tokens de diseño robusto para un Proyecto Fin de Carrera (PFC) encapsula las decisiones de diseño atómicas (colores, espaciados, tipografías) de manera semántica, permitiendo mutabilidad global e interoperabilidad. Asimismo, la API del navegador permite capturar las preferencias del sistema operativo mediante la media-query prefers-color-scheme, habilitando la transición a un modo oscuro nativo sin necesidad de inyectar scripts de JavaScript ni mutar clases en el elemento <body>.

A continuación, se expone el fragmento de código correspondiente a la arquitectura de variables del PFC:

```
:root {  
  /* Colores */  
  --color-primary:      #1557b0; /* Azul oscuro de alto contraste con blanco  
(6.54:1) */  
  --color-primary-dark: #0e3c7d;  
  --color-secondary:    #34a853;  
  --color-danger:       #d93025;  
  --color-surface:      #ffffff;  
  --color-surface-alt:  #f8f9fa;  
  --color-text:         #202124;  
  --color-text-muted:   #5f6368;
```

```

--color-border:      #dadce0;
--color-on-primary:   #ffffff;
--color-header-bg:    #1557b0;

/* Tipografía fluida */
--font-base:    clamp(1rem, 2.5vw, 1.125rem);
--font-lg:      clamp(1.25rem, 3vw, 1.5rem);
--font-xl:      clamp(1.5rem, 4vw, 2rem);
--font-family:  'Segoe UI', system-ui, sans-serif;

/* Espaciado */
--spacing-xs:   0.25rem;
--spacing-sm:   0.5rem;
--spacing-md:   1rem;
--spacing-lg:   1.5rem;
--spacing-xl:   2rem;

/* Bordes y sombras */
--radius-sm:    4px;
--radius-md:    8px;
--radius-lg:    16px;
--shadow-sm:    0 1px 3px rgba(0,0,0,.12);
--shadow-md:    0 4px 12px rgba(0,0,0,.15);

/* Transiciones */
--transition:   0.2s ease-in-out;
}

:root[data-theme="dark"] {
  --color-surface:      #1e1e1e;
  --color-surface-alt:  #2d2d2d;
  --color-text:         #e8eae2;
  --color-text-muted:   #9aa0a6;
  --color-border:       #3c4043;
  --color-primary:      #8ab4f8; /* Azul claro de alto contraste con oscuro (4.55:1)
  */
  --color-primary-dark: #aecbfa;
  --color-on-primary:   #121212; /* Texto oscuro sobre azul claro */
  --color-header-bg:    #2d2d2d; /* Header oscuro en modo oscuro para evitar
  contraste defectuoso */
  --shadow-sm:          0 1px 3px rgba(0,0,0,.5);
  --shadow-md:          0 4px 12px rgba(0,0,0,.6);
}

@media (prefers-color-scheme: dark) {

```

```

:root:not([data-theme="light"]) {
  --color-surface: #1e1e1e;
  --color-surface-alt: #2d2d2d;
  --color-text: #e8eae;
  --color-text-muted: #9aa0a6;
  --color-border: #3c4043;
  --color-primary: #8ab4f8;
  --color-primary-dark: #aecbfa;
  --color-on-primary: #121212;
  --color-header-bg: #2d2d2d;
  --shadow-sm: 0 1px 3px rgba(0,0,0,.5);
  --shadow-md: 0 4px 12px rgba(0,0,0,.6);
}
}

```

3.4. Accesibilidad en CSS

La inclusión de criterios de accesibilidad desde la capa de estilos es un imperativo legal y técnico regido por las pautas WCAG [7]. Los usuarios propensos a crisis vestibulares o con sensibilidades neurológicas requieren la mitigación de estímulos agresivos; esto se solventa mediante la consulta de medios prefers-reduced-motion, la cual detecta si el usuario ha configurado su terminal para minimizar las animaciones:

```

@media (prefers-reduced-motion: reduce) {
  *, ::before, ::after {
    animation-delay: -1ms !important;
    animation-duration: 1ms !important;
    animation-iteration-count: 1 !important;
    scroll-behavior: auto !important;
    transition-duration: 0s !important;
    transition-delay: 0s !important;
  }
}

```

Para el ratio de contraste mínimo de 4.5:1 (WCAG Nivel AA), la verificación programática se realiza calculando la luminancia relativa (L) de los colores en el espacio sRGB [7].

Primero, se extraen las componentes normalizadas de color de 8 bits $C_{srgb} = \frac{C_{8bit}}{255}$ (donde) $C \in \{R, G, B\}$. Posteriormente, se linealizan matemáticamente para contrarrestar la corrección de gamma mediante la siguiente función de transformación:

$$\begin{aligned}
 \text{Si } C_{srgb} \leq 0.03932 &\rightarrow C = \frac{C_{8bit}}{12.92} \\
 \text{De lo contrario } C &= \left(\frac{C_{srgb} + 0.055}{1.055} \right)^{2.4}
 \end{aligned}$$

Una vez obtenidos los coeficientes lineales de los tres canales (R, G, B), la luminancia relativa global (L) del color se determina mediante la ecuación ponderada de la percepción humana:

$$L = 0.2126 * R + 0.7152 * G + 0.0722 * B$$

Finalmente, el ratio de contraste (CR) se define algebraicamente mediante el cociente de las luminancias de los dos tonos en comparación, donde L_1 corresponde al color más claro y L_2 al color más oscuro:

$$CR = \frac{L_1 + 0.05}{L_2 + 0.05}$$

El valor resultante de CR debe satisfacer estrictamente la inecuación $CR \geq 4.5$ para validar el cumplimiento del estándar de accesibilidad web.

4. JavaScript ES6+ y Programación Asíncrona

4.1. El Event Loop y la Naturaleza Asíncrona de JavaScript

JavaScript opera bajo un modelo de concurrencia fundamentalmente distinto al de lenguajes como Java o C#: es un lenguaje de hilo único (single-threaded). El motor V8 posee un único Call Stack de ejecución. Esta característica arquitectónica implica que solo puede ejecutarse una instrucción a la vez dentro del contexto del motor [15].

Las aplicaciones web modernas realizan operaciones inherentemente lentas: peticiones de red con latencias de cientos de milisegundos, operaciones en IndexedDB, temporizadores y procesamiento de eventos. Si JavaScript procesara estas operaciones de forma sincrónica y bloqueante, el hilo de renderizado quedaría congelado, haciendo la interfaz completamente irresponsiva. El mecanismo que resuelve esta contradicción es el Event Loop, que opera en coordinación con las Web APIs del entorno de ejecución del navegador [16].

Cuando JavaScript encuentra una operación asíncrona (`fetch()`, `setTimeout()` o un `EventListener`), la delega al entorno del navegador, que dispone de sus propios hilos de sistema operativo para ejecutarlas de forma concurrente. Una vez que la operación concluye, el resultado se coloca en la cola de tareas apropiada para ser procesado por el Event Loop cuando el Call Stack esté vacío. Esto garantiza que el hilo principal nunca se bloquee esperando una respuesta de red [16].

4.2. Anatomía Componente a Componente del Event Loop

- **Call Stack:** Estructura de datos LIFO que registra el contexto de ejecución activo. Cada llamada a función empuja un Frame al stack; cuando la función retorna, el Frame es expulsado. Si el stack permanece ocupado, el navegador no puede repintar la interfaz, ocasionando el fenómeno conocido como 'jank' o congelamiento visual.
- **Web APIs del Entorno de Ejecución:** Interfaces provistas por el navegador (no por V8) que permiten ejecutar operaciones costosas fuera del hilo principal: Timer API (`setTimeout`, `setInterval`), Fetch API, DOM Event System y Geolocation API. Operan en hilos de sistema operativo independientes y notifican al Event Loop al concluir [15].
- **Callback Queue (Macro-task Queue):** Cola FIFO donde se encolan los callbacks de operaciones Web API finalizadas. El Event Loop despacha una sola macro-tarea por ciclo, únicamente cuando el Call Stack está completamente vacío [17].
- **Microtask Queue:** Cola de mayor prioridad que el Callback Queue, introducida con ES6 para la resolución de Promesas (`.then()`, `.catch()`, `.finally()`) y callbacks de `queueMicrotask()`. El Event Loop drena completamente la Microtask Queue después de cada tarea del Call Stack y antes de procesar la siguiente macro-tarea [17].

Demostración (Pseudocódigo) - `async/await` como azúcar sintáctica: `async/await` no crea un nuevo hilo, sino que pausa la ejecución local de la función devolviendo el control al hilo principal, enmascarando el encadenamiento de una Promesa.

```
// Usando Promesas (Microtask Queue)
```

```
function obtenerDatos() {
  fetch('https://api.com/datos')
    .then(response => response.json())
    .then(data => console.log(data));
  console.log("Esto no se bloquea");
}

// Usando async/await (Azúcar sintáctica, el hilo principal no se bloquea)
async function obtenerDatosAsync() {
  // 'await' cede el control al Event Loop hasta que la promesa se resuelva
  const response = await fetch('https://api.com/datos');
  const data = await response.json();
  console.log(data);
}
console.log("Esto no se bloquea");
```

4.3. Módulos ES6 y Arquitectura Frontend

Evolución Histórica de la Gestión de Dependencias

Antes de la estandarización de los módulos nativos, los desarrolladores usaban el patrón IIFE para crear un scope privado mediante closures. Sin embargo, la gestión de dependencias entre módulos requería orden manual de scripts en el HTML, propenso a errores silenciosos. CommonJS emergió como estándar de facto en Node.js (2009), introduciendo `require()` para carga síncrona. Su semántica síncrona resultó problemática en el navegador, donde el bloqueo congelaría el renderizado. AMD y RequireJS intentaron resolver esto con carga asíncrona, pero a costa de una sintaxis verbosa [18].

El estándar ECMAScript Modules (ESM), formalizado en ES2015, resolvió definitivamente estas limitaciones. Los módulos ESM son estáticos por diseño —sus importaciones son declarativas, no imperativas— lo que permite a los bundlers analizar el grafo de dependencias en tiempo de compilación y eliminar código no utilizado mediante Tree Shaking [15].

Tabla Comparativa de Sistemas de Módulos

Sistema	Mecanismo de Carga	Ventajas de Mantenibilidad	Desventajas / Limitaciones en Navegador
IIFE	Síncrono, orden manual de <code><script></code> en HTML. Sin importación formal.	Encapsulación básica del scope. Sin dependencias externas. Compatible con todos los navegadores.	Sin gestión automática de dependencias. No permite Tree Shaking. Propenso a conflictos en window global.
CommonJS	Síncrono y bloqueante. Diseñado para sistemas de ficheros (Node.js). No nativo en navegadores.	Gestión de dependencias clara. Amplio ecosistema npm. <code>require()</code> dinámico permite carga condicional.	Carga síncrona incompatible con la red. Requiere bundler (Webpack, Browserify). Tree Shaking ineficiente.
ES6 Modules (ESM)	Asíncrono y diferido. Nativo en navegadores modernos con <code>type="module"</code> . Analizable estáticamente.	Dependencias explícitas y estáticas. Soporte nativo sin bundler. Scope aislado. Tree Shaking y Code Splitting.	Incompatible con IE11. Requiere servidor HTTP (no funciona con <code>file://</code>). CORS aplica a importaciones remotas.

Arquitectura del PFC: Separación de Responsabilidades

La arquitectura del PFC adopta el Principio de Responsabilidad Única (SRP) de SOLID, aplicado a la capa frontend. La estructura de módulos propuesta es:

```
src/
├── js/
│   ├── api.js      ← Capa de Infraestructura y Red
│   │               (encapsula toda comunicación con APIs externas,
│   │               manejo de errores HTTP, AbortController)
│   ├── ui.js       ← Capa de Presentación e Interacción DOM
│   │               (manipulación del DOM, renderizado de estados,
│   │               gestión de aria-live, gestión del foco)
│   ├── storage.js  ← CRUD sobre localStorage para registros del formulario
│   └── app.js      ← Orquestador de Aplicación y Flujos
│                   (importa api.js y ui.js, coordina flujos,
│                   registra event listeners)
```

Esta separación produce beneficios cuantificables: (1) testabilidad unitaria de cada módulo de forma aislada; (2) mantenibilidad —un cambio en la URL de la API solo afecta a `api.js`—; (3) legibilidad —un desarrollador nuevo comprende el flujo completo leyendo únicamente `app.js`—; y (4) escalabilidad —nuevas funcionalidades se añaden como módulos sin modificar los existentes [15].

Código de Completo — Cuatro Módulos del PFC

Archivo: `src/js/api.js` (Capa de Infraestructura y Red)

```
export class HttpError extends Error {
  constructor(status, statusText, servicio) {
    super(`${servicio}: ${status} ${statusText}`);
    this.name = 'HttpError';
    this.status = status;
    this.statusText = statusText;
    this.servicio = servicio;
  }
}

const NASA_KEY = 'omVTC25hm9VTfRxuTninsIHPzybF0Xoqr2itqQJN'; // o
'DEMO_KEY' para pruebas
const GOOGLE_WEATHER_KEY = 'AQ.Ab8RN6IqkgRCz0u6czfrE9IrnTu0atPH0bihf23Lm1s54_OhAw';
const LAT = -2.1894;
const LON = -79.8891;

export async function fetchCoffee() {
```



```

return new Promise((resolve) => {
  setTimeout(() => {
    resolve({ file: `https://coffee.alexflipnote.dev/random?t=${Date.now()}` });
  }, 500);
});
}

```

```

export async function fetchNASA(date = null) {
  let url = `https://api.nasa.gov/planetary/apod?api_key=${NASA_KEY}`;
  if (date) {
    url += `&date=${date}`;
  }
  const res = await fetch(url);
  if (!res.ok) {
    throw new HttpError(res.status, res.statusText, 'NASA APOD');
  }
  const data = await res.json();
  if (data && data.url) {
    data.url = data.url.replace('http://', 'https://');
  }
  return data;
}

```

```

export async function fetchPhrase() {
  const url = `https://www.positive-api.online/phrase/esp`;
  const res = await fetch(url);
  if (!res.ok) {
    throw new HttpError(res.status, res.statusText, 'Frase Positiva');
  }
  return res.json();
}

```

Archivo: src/js/ui.js (Capa de Presentación e Interacción DOM)

```

import { eliminarRegistro, obtenerUbicaciones } from './storage.js';
import { HttpError } from './api.js';

```

```

export function clasificarError(err) {
  if (err.name === 'HttpError' || err instanceof HttpError) {
    const status = err.status;
    if (status >= 400 && status < 500) {
      return `Error de Solicitud (${status}): No se pudo obtener la información (Ej. recurso no encontrado o sin autorización).`;
    }
  }
}

```

```

    } else if (status >= 500) {
      return `Error de Servidor (${status}): El servidor de la API experimenta
      problemas. Intente más tarde.`;
    }
    return `Error HTTP (${status}): ${err.statusText}`;
  }
  if (err instanceof TypeError) {
    return 'Fallo de Red: No se pudo conectar al servidor. Verifique su conexión de
    red o restricciones CORS.';
  }
  return `Error inesperado: ${err.message}`;
}

```

// — Utilidades de UI

```

/** Muestra u oculta el spinner de carga dentro de un contenedor. */
export function toggleSpinner(contenedorId, mostrar) {
  const targetId = document.getElementById(`${contenedorId}-content`) ?
  `${contenedorId}-content` : contenedorId;
  const el = document.getElementById(targetId);
  if (!el) return;
  if (mostrar) {
    el.innerHTML = '<div class="spinner" role="status" aria-
    label="Cargando..."></div>';
  }
}

```

```

/** Muestra un mensaje de error accesible dentro de un contenedor. */
export function mostrarError(contenedorId, mensaje) {
  const targetId = document.getElementById(`${contenedorId}-content`) ?
  `${contenedorId}-content` : contenedorId;
  const el = document.getElementById(targetId);
  if (!el) return;
  if (contenedorId === 'widget-nasa') {
    const btnHTML = `<button id="btn-cambiar-nasa" style="display: block; margin: 0
    auto var(--spacing-sm); padding: 0.3rem 0.8rem; font-size:
    0.9rem;">Cambiar</button>`;
    el.innerHTML = `
      ${btnHTML}
      <p class="error-msg" role="alert" style="margin-top:var(--spacing-md); text-
      align:center;">⚠️ ${mensaje}</p>
    `;
    return;
  }
  if (contenedorId === 'widget-coffee') {

```

```

    const btnHTML = `<button id="btn-cambiar-coffee" style="display: block; margin:
0 auto var(--spacing-sm); padding: 0.3rem 0.8rem; font-size:
0.9rem;">Cambiar</button>`;
    el.innerHTML = `
        ${btnHTML}
        <p class="error-msg" role="alert" style="margin-top:var(--spacing-md); text-
align:center;">⚠️ ${mensaje}</p>
    `;
    return;
}
el.innerHTML = `<p class="error-msg" role="alert">⚠️ ${mensaje}</p>`;
}

```

// — Renderizado de APIs —————

```

/** Renderiza la imagen de café en #widget-coffee */
export function renderCoffee(datos) {
    const el = document.getElementById('widget-coffee-content') ||
document.getElementById('widget-coffee');
    if (!el) return;
    const btnHTML = `<button id="btn-cambiar-coffee" style="display: block; margin: 0
auto var(--spacing-sm); padding: 0.3rem 0.8rem; font-size:
0.9rem;">Cambiar</button>`;
    el.innerHTML = `
        ${btnHTML}
        
    `;
}

```

```

/** Renderiza la imagen NASA APOD en #widget-nasa */
export function renderNASA(datos) {
    const el = document.getElementById('widget-nasa-content') ||
document.getElementById('widget-nasa');
    if (!el) return;
    const btnHTML = `<button id="btn-cambiar-nasa" style="display: block; margin: 0
auto var(--spacing-sm); padding: 0.3rem 0.8rem; font-size:
0.9rem;">Cambiar</button>`;
    if (datos.media_type === 'image') {
        el.innerHTML = `
            ${btnHTML}

```

```

        
        <p style="margin-top:var(--spacing-xs);">${datos.explanation.slice(0,
200)}...</p>
    `;
    } else {
        el.innerHTML = `
            ${btnHTML}
            <p style="margin-top:var(--spacing-xs);">Hoy el contenido es un video. <a
href="${datos.url}" target="_blank" rel="noopener">Ver en NASA</a></p>`;
    }
}

/** Renderiza la frase positiva en #widget-phrase */
export function renderPhrase(datos) {
    const el = document.getElementById('widget-phrase-content') ||
document.getElementById('widget-phrase');
    if (!el) return;
    const fraseTexto = typeof datos === 'string' ? datos : (datos.phrase || datos.text
|| datos.mensaje || JSON.stringify(datos));
    el.innerHTML = `
        <blockquote style="font-style: italic; border-left: 4px solid var(--color-
primary); padding-left: var(--spacing-sm); margin-top: var(--spacing-md);">
            "${fraseTexto}"
        </blockquote>
    `;
}

// — Gestión del Modal Accesible (Focus Trap) —————

let registroAEliminarId = null;
let botonDisparador = null;
let releaseTrap = null;
let modalInitialized = false;

const FOCUSABLE = [
    'a[href]', 'button:not([disabled])', 'input:not([disabled])',
    'select:not([disabled])', 'textarea:not([disabled])',
    '[tabindex]:not([tabindex="-1"])]'
].join(', ');

export function trapFocus(modal) {

```

```

const els = Array.from(modal.querySelectorAll(FOCUSABLE));
if (els.length === 0) return () => {};
const first = els[0], last = els[els.length - 1];
first.focus();

function handleKeyDown(e) {
  if (e.key !== 'Tab') return;
  if (e.shiftKey) {
    if (document.activeElement === first) {
      e.preventDefault(); last.focus();
    }
  } else {
    if (document.activeElement === last) {
      e.preventDefault(); first.focus();
    }
  }
}

function handleEscape(e) {
  if (e.key === 'Escape') {
    modal.dispatchEvent(new CustomEvent('modal:close'));
  }
}

modal.addEventListener('keydown', handleKeyDown);
modal.addEventListener('keydown', handleEscape);
return () => {
  modal.removeEventListener('keydown', handleKeyDown);
  modal.removeEventListener('keydown', handleEscape);
};
}

export function abrirModal(modal, activador) {
  modal.removeAttribute('hidden');
  modal.setAttribute('aria-modal', 'true');
  releaseTrap = trapFocus(modal);
}

export function cerrarModal(modal, activador) {
  if (releaseTrap) {
    releaseTrap();
    releaseTrap = null;
  }
  modal.setAttribute('hidden', '');
  if (activador && document.body.contains(activador)) {
    activador.focus();
  } else {

```

```

    const tabla = document.getElementById('tabla-registros');
    if (tabla) {
        tabla.focus();
    }
}

function ocultarModalEliminar() {
    const modal = document.getElementById('modal-confirmacion');
    if (modal) {
        cerrarModal(modal, botonDisparador);
    }
}

function initModallListeners() {
    if (modalInitialized) return;
    modalInitialized = true;

    const btnConfirmar = document.getElementById('modal-btn-confirmar');
    const btnCancelar = document.getElementById('modal-btn-cancelar');
    const modal = document.getElementById('modal-confirmacion');

    if (btnConfirmar) {
        btnConfirmar.addEventListener('click', () => {
            if (registroAEliminarId !== null) {
                const actualizados = eliminarRegistro(registroAEliminarId);
                renderTablaRegistros(actualizados);
            }
            ocultarModalEliminar();
        });
    }

    if (btnCancelar) {
        btnCancelar.addEventListener('click', ocultarModalEliminar);
    }

    if (modal) {
        modal.addEventListener('modal:close', ocultarModalEliminar);
    }
}

window.eliminar = (id) => {
    registroAEliminarId = id;
    botonDisparador = document.activeElement;
};

```

```

const modal = document.getElementById('modal-confirmacion');
if (modal) {
  abrirModal(modal, botonDisparador);
}
};

// — Tabla de registros —————

/** Renderiza la tabla de registros. */
export function renderTablaRegistros(registros) {
  const tbody = document.getElementById('tbody-registros');
  const vacio = document.getElementById('estado-vacio');
  const contador = document.getElementById('contador-registros');

  contador.textContent = `${registros.length} registro${registros.length !== 1 ? 's'
: ''}`;

  if (registros.length === 0) {
    tbody.innerHTML = '';
    vacio.hidden = false;
    return;
  }
  vacio.hidden = true;
  tbody.innerHTML = registros.map((r, i) => `
    <tr>
      <td>${i + 1}</td>
      <td>${r.nombre ?? '-'}</td>
      <td>${r.email ?? '-'}</td>
      <td>${r.telefono ?? '-'}</td>
      <td>${r.fecha_nacimiento ?? '-'}</td>
      <td>${r.provincia ?? '-'}</td>
      <td>${r.nivel ?? '-'}</td>
      <td>
        <button onclick="eliminar(${r.id})"
          aria-label="Eliminar registro de ${r.nombre}">

      </button>
    </td>
    </tr>
  `).join('');

  // Asegurar que los listeners del modal estén registrados
  initModalListeners();
}

```

```

// — Leaflet Map Integración
let map = null;
let currentCoords = null;
let leafletMarkers = [];

export function renderUbicaciones(ubicaciones) {
  const list = document.getElementById('markedLocations');
  if (!list) return;

  // Limpiar marcadores antiguos del mapa
  if (map) {
    leafletMarkers.forEach(m => map.removeLayer(m));
  }
  leafletMarkers = [];

  list.innerHTML = '';

  if (ubicaciones.length === 0) {
    list.innerHTML = '<li class="estado-vacio" style="text-align: center; width: 100%;">No hay ubicaciones registradas. Haz clic en el mapa para añadir una.</li>';
    return;
  }

  ubicaciones.forEach((ubicacion) => {
    const li = document.createElement('li');
    li.className = 'marked-location-item';

    li.innerHTML = `
      <span>📍 ${ubicacion.name}</span>
      <button class="btn-ver-ubicacion" data-lat="${ubicacion.lat}" data-lng="${ubicacion.lng}">Ver</button>
    `;
    list.appendChild(li);

    if (map) {
      const m = L.marker([ubicacion.lat, ubicacion.lng])
        .addTo(map)
        .bindPopup(`<b>${ubicacion.name}</b><br>Lat:
        ${parseFloat(ubicacion.lat).toFixed(4)} | Lng:
        ${parseFloat(ubicacion.lng).toFixed(4)}`);
      leafletMarkers.push(m);
    }
  });

  const botones = list.querySelectorAll('.btn-ver-ubicacion');

```



```

botones.forEach(btn => {
  btn.addEventListener('click', (e) => {
    const lat = parseFloat(e.target.getAttribute('data-lat'));
    const lng = parseFloat(e.target.getAttribute('data-lng'));
    if (map) {
      map.panTo(new L.LatLng(lat, lng));
      leafletMarkers.forEach(m => {
        const latLng = m.getLatLng();
        if (Math.abs(latLng.lat - lat) < 0.00001 && Math.abs(latLng.lng - lng) <
0.00001) {
          m.openPopup();
        }
      });
    }
  });
});
}

```

```

export function initMap() {
  const mapElement = document.getElementById('map');
  if (!mapElement) return;

  // Centro inicial en el Campus de la UTEQ en Quevedo (-1.0125, -79.4696)
  map = L.map('map').setView([-1.0125, -79.4696], 16);

  L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: '&copy; <a
href="https://www.openstreetmap.org/copyright">OpenStreetMap</a> contributors'
  }).addTo(map);

  const ubicaciones = obtenerUbicaciones();
  renderUbicaciones(ubicaciones);

  map.on('click', (e) => {
    currentCoords = e.latlng;
    const panel = document.getElementById('marked-container');
    if (panel) {
      panel.style.display = 'block';
      document.getElementById('latitude').value = currentCoords.lat.toFixed(6);
      document.getElementById('longitude').value = currentCoords.lng.toFixed(6);
      document.getElementById('markerName').value = '';
      document.getElementById('markerName').focus();
    }
  });
}

```

Archivo: src/js/storage.js (Crud de la tabla de registros)

// js/storage.js – Módulo: CRUD sobre localStorage para registros del formulario

```
const CLAVE = 'pfc_registros';
const CLAVE_MAPA = 'pfc_ubicaciones';

/** Devuelve el array de registros guardados. */
export function obtenerRegistros() {
  try {
    return JSON.parse(localStorage.getItem(CLAVE)) ?? [];
  } catch {
    return [];
  }
}

/** Guarda un nuevo registro. Retorna el array actualizado. */
export function guardarRegistro(datos) {
  const registros = obtenerRegistros();
  registros.push({ ...datos, id: Date.now() });
  localStorage.setItem(CLAVE, JSON.stringify(registros));
  return registros;
}

/** Elimina un registro por id. */
export function eliminarRegistro(id) {
  const actualizados = obtenerRegistros().filter(r => r.id !== id);
  localStorage.setItem(CLAVE, JSON.stringify(actualizados));
  return actualizados;
}

/** Limpia todos los registros. */
export function limpiarTodo() {
  localStorage.removeItem(CLAVE);
  return [];
}

/** Devuelve las ubicaciones del mapa guardadas. */
export function obtenerUbicaciones() {
  try {
    return JSON.parse(localStorage.getItem(CLAVE_MAPA)) ?? [];
  } catch {
    return [];
  }
}
```

```

/** Guarda una nueva ubicación del mapa. Retorna el array actualizado. */
export function guardarUbicacion(ubicacion) {
  const ubicaciones = obtenerUbicaciones();
  ubicaciones.push({ ...ubicacion, id: Date.now() });
  localStorage.setItem(CLAVE_MAPA, JSON.stringify(ubicaciones));
  return ubicaciones;
}

/** Limpia todas las ubicaciones del mapa. */
export function limpiarUbicaciones() {
  localStorage.removeItem(CLAVE_MAPA);
  return [];
}

```

Archivo: src/js/app.js (Orquestador de Aplicación)

```

import { fetchCoffee, fetchNASA, fetchPhrase } from './api.js';
import { guardarRegistro, obtenerRegistros, limpiarTodo, guardarUbicacion } from
'./storage.js';
import {
  toggleSpinner, mostrarError,
  renderCoffee, renderNASA, renderPhrase,
  renderTablaRegistros, clasificarError,
  initMap, renderUbicaciones
} from './ui.js';

```

// — Cargar APIs al iniciar

```

async function cargarAPIs() {
  // Coffee
  toggleSpinner('widget-coffee', true);
  try {
    const coffee = await fetchCoffee();
    renderCoffee(coffee);
  } catch (err) {
    const msg = clasificarError(err);
    mostrarError('widget-coffee', msg);
    console.error(err);
  }

  // NASA APOD
  toggleSpinner('widget-nasa', true);
  try {
    const nasa = await fetchNASA();
    renderNASA(nasa);
  } catch (err) {

```

```

    const msg = clasificarError(err);
    mostrarError('widget-nasa', msg);
    console.error(err);
}

// Frase Positiva
toggleSpinner('widget-phrase', true);
try {
    const frase = await fetchPhrase();
    renderPhrase(frase);
} catch (err) {
    const msg = clasificarError(err);
    mostrarError('widget-phrase', msg);
    console.error(err);
}
}

// — Slider nivel — actualizar output en tiempo real —————

function initSlider() {
    const slider = document.getElementById('nivel');
    const output = document.getElementById('nivel-valor');
    if (!slider || !output) return;
    slider.addEventListener('input', () => { output.textContent = slider.value; });
}

// — Formulario — envío y validación —————

function validarCampo(input) {
    const descripcion = input.getAttribute('aria-describedby');
    if (!descripcion) return true;

    const errorId = descripcion.split(' ').find(id => id.endsWith('-error'));
    if (!errorId) return true;

    const errorSpan = document.getElementById(errorId);
    if (!errorSpan) return true;

    if (input.validity.valid) {
        errorSpan.textContent = '';
        input.classList.remove('is-invalid');
        input.setAttribute('aria-invalid', 'false');
        return true;
    }
}

```

```

let mensaje = 'Campo inválido.';

if (input.validity.valueMissing) {
  mensaje = 'Este campo es obligatorio.';
} else if (input.validity.tooShort) {
  mensaje = `Debe tener al menos ${input.minLength} caracteres.`;
} else if (input.validity.tooLong) {
  mensaje = `No puede superar los ${input.maxLength} caracteres.`;
} else if (input.validity.typeMismatch && input.type === 'email') {
  mensaje = 'Por favor, ingrese un correo electrónico válido.';
} else if (input.validity.patternMismatch) {
  if (input.id === 'password') {
    mensaje = 'Mínimo 8 caracteres, una mayúscula y un número.';
  } else if (input.id === 'telefono') {
    mensaje = 'El teléfono debe contener entre 7 y 15 números.';
  } else {
    mensaje = 'Formato incorrecto.';
  }
} else if (input.validity.rangeUnderflow) {
  mensaje = `El valor mínimo es ${input.min}.`;
} else if (input.validity.rangeOverflow) {
  if (input.id === 'fecha-nacimiento') {
    mensaje = 'Debes ser mayor de edad (nacido antes del 2008-01-01).';
  } else {
    mensaje = `El valor máximo es ${input.max}.`;
  }
} else {
  mensaje = input.validationMessage;
}

errorSpan.textContent = mensaje;
input.classList.add('is-invalid');
input.setAttribute('aria-invalid', 'true');
return false;
}

function initFormulario() {
  const form = document.getElementById('form-registro');
  if (!form) return;

  const campos = form.querySelectorAll('input, select, textarea');

  // Validación interactiva en tiempo real
  campos.forEach(campo => {
    campo.addEventListener('input', () => {

```

```

        validarCampo(campo);
    });
    campo.addEventListener('change', () => {
        validarCampo(campo);
    });
});

// Limpiar clases y mensajes de error al resetear
form.addEventListener('reset', () => {
    campos.forEach(campo => {
        campo.classList.remove('is-invalid');
        campo.setAttribute('aria-invalid', 'false');
        const descripcion = campo.getAttribute('aria-describedby');
        if (descripcion) {
            const errorId = descripcion.split(' ').find(id => id.endsWith('-error'));
            if (errorId) {
                const errorSpan = document.getElementById(errorId);
                if (errorSpan) errorSpan.textContent = '';
            }
        }
    });
    document.getElementById('nivel-valor').textContent = '5';
});

form.addEventListener('submit', (e) => {
    e.preventDefault();

    let formValido = true;
    let primerInvalido = null;

    // Validar todos los campos antes de enviar
    campos.forEach(campo => {
        const valido = validarCampo(campo);
        if (!valido) {
            formValido = false;
            if (!primerInvalido) {
                primerInvalido = campo;
            }
        }
    });

    if (!formValido) {
        if (primerInvalido) {
            primerInvalido.focus();
        }
    }
});

```

```

        return;
    }

    const datos = Object.fromEntries(new FormData(form));
    guardarRegistro(datos);
    renderTablaRegistros(obtenerRegistros());
    form.reset();

    // Anunciar éxito a lectores de pantalla
    const aviso = document.createElement('div');
    aviso.setAttribute('role', 'status');
    aviso.setAttribute('aria-live', 'polite');
    aviso.textContent = 'Registro guardado correctamente.';
    document.body.appendChild(aviso);
    setTimeout(() => aviso.remove(), 3000);
  });
}

// — Limpiar todos los registros —————

function initBtnLimpiar() {
  const btn = document.getElementById('btn-limpiar-registros');
  if (!btn) return;
  btn.addEventListener('click', () => {
    if (confirm('¿Eliminar todos los registros?')) {
      renderTablaRegistros(limpiarTodo());
    }
  });
}

// — Theme Toggle —————

function initThemeToggle() {
  const btnTheme = document.getElementById('btn-theme');
  if (!btnTheme) return;

  const currentTheme = localStorage.getItem('theme');
  if (currentTheme) {
    document.documentElement.setAttribute('data-theme', currentTheme);
  }

  btnTheme.addEventListener('click', () => {
    let theme = document.documentElement.getAttribute('data-theme');
    if (!theme) {
      const prefersDark = window.matchMedia('(prefers-color-scheme: dark)').matches;

```

```

    theme = prefersDark ? 'dark' : 'light';
  }
  const newTheme = theme === 'dark' ? 'light' : 'dark';
  document.documentElement.setAttribute('data-theme', newTheme);
  localStorage.setItem('theme', newTheme);
});
}

```

// — Mapa de ubicaciones – guardar y cerrar panel —————

```

function initMapaEventos() {
  const btnClose = document.getElementById('close-marker-panel');
  const btnSave = document.getElementById('saveMarker');
  const panel = document.getElementById('marked-container');
  const inputName = document.getElementById('markerName');

  if (btnClose && panel) {
    btnClose.addEventListener('click', () => {
      panel.style.display = 'none';
      if (inputName) inputName.value = '';
    });
  }

  if (btnSave && panel) {
    btnSave.addEventListener('click', () => {
      if (!inputName) return;
      const name = inputName.value.trim();
      const lat = document.getElementById('latitude').value;
      const lng = document.getElementById('longitude').value;

      if (!name) {
        alert('Por favor, ingrese un nombre para el área.');
```

inputName.focus();

```

        return;
      }

      const nuevaUbi = { name, lat, lng };
      const actualizadas = guardarUbicacion(nuevaUbi);
      renderUbicaciones(actualizadas);

      panel.style.display = 'none';
      inputName.value = '';
    });
  }
}

```


// — Inicio —————

```
document.addEventListener('DOMContentLoaded', () => {
  initThemeToggle();
  renderTablaRegistros(obtenerRegistros()); // cargar registros previos
  initSlider();
  initFormulario();
  initBtnLimpiar();
  initMap(); // Iniciar mapa interactivo Leaflet
  initMapaEventos(); // Iniciar eventos del panel de marcador
  cargarAPIs();
});
```

// — Event Delegation para widgets —————

```
document.addEventListener('click', async (e) => {
  if (e.target && e.target.id === 'btn-cambiar-nasa') {
    // Generar fecha aleatoria entre 1995-06-16 (fecha inicio APOD) y hoy
    const start = new Date(1995, 5, 16).getTime();
    const end = new Date().getTime();
    const randomDate = new Date(start + Math.random() * (end - start));
    const yyyy = randomDate.getFullYear();
    const mm = String(randomDate.getMonth() + 1).padStart(2, '0');
    const dd = String(randomDate.getDate()).padStart(2, '0');
    const dateStr = `${yyyy}-${mm}-${dd}`;

    toggleSpinner('widget-nasa', true);
    try {
      const nasa = await fetchNASA(dateStr);
      renderNASA(nasa);
    } catch (err) {
      const msg = clasificarError(err);
      mostrarError('widget-nasa', msg);
      console.error(err);
    }
  }

  if (e.target && e.target.id === 'btn-cambiar-coffee') {
    toggleSpinner('widget-coffee', true);
    try {
      const coffee = await fetchCoffee();
      renderCoffee(coffee);
    } catch (err) {
      mostrarError('widget-coffee', `No se pudo cargar el café: ${err.message}`);
    }
  }
});
```

```
}  
});
```

4.4. Manejo Robusto de Errores en Peticiones Fetch

Mecánica de la API Fetch: El Error Silencioso

La Fetch API presenta un comportamiento contraintuitivo que es fuente habitual de defectos en producción: la Promesa retornada por `fetch()` solo transiciona al estado `rejected` ante un fallo de red físico, no ante respuestas HTTP de error. Una respuesta 404 o 500 resuelve la Promesa con estado `fulfilled`, entregando un objeto `Response` aparentemente válido [19].

La propiedad `response.ok` es un booleano que evalúa `true` únicamente para códigos 200-299, y `false` para cualquier otro código. Su verificación es obligatoria en todo código de producción [18].

Tipificación del Fallo: Taxonomía de Errores

Error de Red Físico (TypeError): El navegador no puede establecer conexión TCP (sin Internet, DNS fallido, puerto cerrado). La Promesa se rechaza con `TypeError`.

Bloqueo por Política CORS: El servidor no incluye cabeceras `Access-Control-Allow-Origin`. El navegador bloquea la respuesta lanzando `TypeError`, indistinguible de un error de red desde el cliente [17].

Errores de Cliente (4xx): 400, 401, 403, 404, 429. La respuesta llega con `response.ok = false`. La aplicación debe informar al usuario según el tipo específico.

Errores de Servidor (5xx): 500, 502, 503. Indican fallo en la infraestructura. La aplicación debería implementar reintentos con `backoff` exponencial.

Excepción en Deserialización (SyntaxError): `response.json()` lanza `SyntaxError` si el `body` no es JSON válido (ej: una página HTML de error del servidor).

Patrón de Implementación Resiliente

Ejemplo realizado:

```
export async function fetchNASA(date = null) {  
  let url = `https://api.nasa.gov/planetary/apod?api_key=${NASA_KEY}`;  
  if (date) {  
    url += `&date=${date}`;  
  }  
  const res = await fetch(url);  
  
  // Equivalente al paso 2: verificación manual de response.ok  
  if (!res.ok) {  
    throw new HttpError(res.status, res.statusText, 'NASA APOD');  
  }  
}
```

```

// Equivalente al paso 3: conversión de formato
const data = await res.json();
if (data && data.url) {
  data.url = data.url.replace('http://', 'https://');
}
return data;
}

// Para manejos de errores
export function clasificarError(err) {
  // Equivalente al control de respuestas HTTP (pasos 4.4, 2.1, 2.2)
  if (err.name === 'HttpError' || err instanceof HttpError) {
    const status = err.status;
    if (status >= 400 && status < 500) {
      return `Error de Solicitud (${status}): No se pudo obtener la información (Ej. recurso no encontrado o sin autorización).`;
    } else if (status >= 500) {
      return `Error de Servidor (${status}): El servidor de la API experimenta problemas. Intente más tarde.`;
    }
    return `Error HTTP (${status}): ${err.statusText}`;
  }

  // Equivalente al control de fallos físicos de red / restricciones CORS (paso 4.2)
  if (err instanceof TypeError) {
    return 'Fallo de Red: No se pudo conectar al servidor. Verifique su conexión de red o restricciones CORS.';
  }

  // Equivalente para cualquier otro error imprevisto (ej. SyntaxError / AbortError)
  return `Error inesperado: ${err.message}`;
}

// Como resultado
// NASA APOD
toggleSpinner('widget-nasa', true);
try {
  const nasa = await fetchNASA();
  renderNASA(nasa);
} catch (err) {
  // Clasifica el error robustamente según su tipo
  const msg = clasificarError(err);
  // Inyecta el error de forma accesible (aria-live/alert)
  mostrarError('widget-nasa', msg);
  console.error(err);
}

```

}

Tabla Comparativa: Fetch API vs. Axios

Criterio	Fetch API (nativa)	Axios (librería externa)
Dependencias externas	Ninguna — integrada en navegadores modernos. 0 KB adicionales.	~14 KB minificado + gzipped. Requiere npm install o CDN.
Errores HTTP nativo	NO — verificación manual de response.ok obligatoria. Promesa resuelta para 4xx/5xx.	SÍ — rechaza automáticamente la Promesa para códigos fuera de 2xx.
Interceptores	NO nativo — requiere wrappers manuales o patrón decorador.	SÍ — axios.interceptors.request/response. Ideal para tokens de autenticación y logging.
Transformación de tipos	NO — debe llamar response.json(), response.text() o response.blob() explícitamente.	SÍ — parsea JSON automáticamente. Configurable vía transformRequest/transformResponse.
Cancelación	AbortController + AbortSignal (estándar W3C). Bien soportado en navegadores modernos.	CancelToken (deprecado) o AbortController desde Axios 0.22+.
Soporte IE11	No disponible en IE11. Requiere polyfill. No relevante para entornos modernos.	Compatible con IE11 mediante XMLHttpRequest interno. Soporte universal.
Lighthouse Performance	0 KB de bundle extra. Máximo puntaje teórico en métricas de carga.	~14 KB adicionales. Penaliza métricas de Unused JavaScript.

Veredicto de Ingeniería para el PFC: La Fetch API nativa es la alternativa óptima. Elimina la deuda técnica asociada a dependencias externas, maximiza las métricas de Performance de Lighthouse y obliga al equipo a implementar manejo de errores explícito, desarrollando comprensión profunda del protocolo HTTP. La ausencia de automatización —que en producción podría justificar Axios— es aquí una ventaja pedagógica.

4.5. Accesibilidad Dinámica con JavaScript (Garantía WCAG 2.1)

La Problemática del DOM Reactivo para Tecnologías Asistivas

Las aplicaciones SPA presentan un desafío estructural para la accesibilidad: el contenido de la interfaz cambia dinámicamente sin recarga de página, y las tecnologías asistivas como NVDA, JAWS o VoiceOver no son notificadas automáticamente de estos cambios [3]. Cuando JavaScript inyecta nuevos elementos en el DOM, el lector de pantalla permanece en el elemento donde estaba posicionado, ajeno al nuevo contenido. Cuando un modal se cierra y el elemento que lo disparó es destruido o reemplazado, el foco queda en un punto indeterminado del documento.

El incumplimiento viola directamente los criterios WCAG 2.1 Nivel AA: Criterio 4.1.3 (Mensajes de Estado), Criterio 2.4.3 (Orden del Foco) y Criterio 2.1.1 (Teclado). En el contexto del PFC, estos fallos producirán penalizaciones en la auditoría Lighthouse Accessibility, haciendo imposible alcanzar el umbral de ≥ 90 [7], [18].

Inclusión Predictiva con ARIA Live Regions

El atributo `aria-live` designa una región del DOM como 'región viva': el lector de pantalla monitoriza los cambios en su contenido y anuncia las actualizaciones de forma proactiva, sin requerir que el foco esté posicionado en esa región [10]:

`aria-live="polite"`: Espera hasta que el usuario termine su interacción actual. Apropiado para confirmaciones de carga exitosa, resultados de búsqueda y actualizaciones de contadores.

`aria-live="assertive"`: Interrumpe inmediatamente cualquier anuncio en curso. Reservar para mensajes de alta urgencia: errores críticos, alertas de seguridad.

`role="status"` y `role="alert"`: Roles semánticos que implícitamente configuran la región como `aria-live="polite"` y `aria-live="assertive"` respectivamente.

```
<div class="campo-grupo">
  <label for="nombre">Nombre completo *</label>
  <input type="text" id="nombre" name="nombre" required
    aria-describedby="nombre-error"
    aria-invalid="false" />
  <span id="nombre-error" class="error-msg" role="status" aria-live="polite"></span>
</div>
/* Ocultación visual de utilidades de accesibilidad */
.skip-link {
  position: absolute;
  left: -9999px;
}
.skip-link:focus {
  left: 1rem;
  top: 1rem;
  z-index: 9999;
  padding: var(--spacing-sm) var(--spacing-md);
  background: var(--color-primary);
  color: #fff;
  border-radius: var(--radius-sm);
}
// Anunciar éxito a lectores de pantalla de forma dinámica (Equivalente a
announceToScreenReader)
const aviso = document.createElement('div');
aviso.setAttribute('role', 'status');
aviso.setAttribute('aria-live', 'polite');
aviso.textContent = 'Registro guardado correctamente.';
document.body.appendChild(aviso);
setTimeout(() => aviso.remove(), 3000); // Se remueve para limpiar el DOM
```

Ingeniería de Gestión del Foco y Focus Trap

La gestión programática del foco es un requisito técnico no negociable en aplicaciones SPA accesibles. Cuando la interfaz cambia significativamente, el foco debe transferirse deliberadamente al elemento más relevante para el contexto nuevo. El método `element.focus()` realiza esta transferencia; `tabindex="-1"` hace un elemento focusable programáticamente sin incluirlo en el orden de tabulación natural; `tabindex="0"` lo incluye en el flujo natural [10].

El Focus Trap garantiza que el foco del teclado no pueda escapar de un componente modal mientras está abierto. Al detectar Tab (o Shift+Tab), evalúa si el foco está en el primer o último elemento focusable del modal y redirige al extremo opuesto, creando un ciclo cerrado de navegación:

```
const FOCUSABLE = [
  'a[href]', 'button:not([disabled])', 'input:not([disabled])',
  'select:not([disabled])', 'textarea:not([disabled])',
  '[tabindex]:not([tabindex="-1"])'
].join(', ');
```

```
export function trapFocus(modal) {
  const els = Array.from(modal.querySelectorAll(FOCUSABLE));
  if (els.length === 0) return () => {};
  const first = els[0], last = els[els.length - 1];
  first.focus();
```

```
function handleKeyDown(e) {
  if (e.key !== 'Tab') return;
  if (e.shiftKey) {
    if (document.activeElement === first) {
      e.preventDefault(); last.focus();
    }
  } else {
    if (document.activeElement === last) {
      e.preventDefault(); first.focus();
    }
  }
}
```

```
function handleEscape(e) {
  if (e.key === 'Escape') {
    modal.dispatchEvent(new CustomEvent('modal:close'));
  }
}
```

```
modal.addEventListener('keydown', handleKeyDown);
```

```

modal.addEventListener('keydown', handleEscape);
return () => {
  modal.removeEventListener('keydown', handleKeyDown);
  modal.removeEventListener('keydown', handleEscape);
};
}

export function abrirModal(modal, activador) {
  modal.removeAttribute('hidden');
  modal.setAttribute('aria-modal', 'true');
  releaseTrap = trapFocus(modal);
}

export function cerrarModal(modal, activador) {
  if (releaseTrap) {
    releaseTrap();
    releaseTrap = null;
  }
  modal.setAttribute('hidden', '');
  // Devuelve el foco al botón disparador original (o a la tabla si este fue
  // eliminado)
  if (activador && document.body.contains(activador)) {
    activador.focus();
  } else {
    const tabla = document.getElementById('tabla-registros');
    if (tabla) {
      tabla.focus();
    }
  }
}

```

5. Resultados de la practica

Verificación del código

En el siguiente repositorio se encuentra alojado el código

https://github.com/JuniorSoft363/Praticas_Experimentales_Appweb.git

5.1. Estructura semántica y Accesible

La arquitectura del documento se ha diseñado siguiendo las directrices del estándar WHATWG. Se implementa un árbol del DOM limpio que expone un árbol de accesibilidad óptimo para tecnologías de

asistencia como NVDA o JAWS. La página principal utiliza una estructura semántica única mediante los elementos <header>, <nav>, <main>, <article>, <section>, <aside> y <footer>

5.2. Formulario HTML5 con 8 tipos de input

Tabla Resumen — Tipos de Input Implementados

#	Tipo	Campo / ID	Obligatorio	Activación móvil	Validaciones principales
1	text	Nombre completo (#nombre)	Sí	Teclado alfanumérico estándar	required, minlength="3", maxlength="60"
2	email	Correo electrónico (#email)	Sí	Teclado con acceso directo a '@' y '.'	required, validación de formato nativa (@, dominio)
3	password	Contraseña (#password)	Sí	Teclado estándar con opción de visibilidad	required, minlength="8", pattern (1 mayúscula + 1 número mínimo)
4	tel	Teléfono (#telefono)	No	Activa teclado numérico en Android e iOS (sin signos matemáticos)	pattern="[0-9]{7,15}" (7 a 15 dígitos)
5	date	Fecha de nacimiento (#fecha-nacimiento)	Sí	Selector nativo de fecha del sistema operativo (optimizado para táctil)	required, max="2008-01-01" (mayor de edad)
6	number	Edad (#edad)	Sí	Teclado numérico con flechas de incremento/decremento	required, min="16", max="120"
7	range	Nivel de experiencia (#nivel)	Sí	Control deslizante táctil nativo	min="1", max="10", value="5" (valor por defecto)
8	file	Foto de perfil (#avatar)	No	Abre selector de archivos o cámara del dispositivo	accept="image/*" (filtro de tipo MIME nativo)
9	radio	Género (grupo) (#genero-*)	No	Botones de opción táctiles nativos del sistema	Agrupación por name="genero"; selección mutuamente excluyente nativa
10	checkbox	Aceptar términos (#terminos)	Sí	Casilla táctil nativa con área de toque amplia	required (el formulario no envía si no está marcado)
11	select	Provincia (#provincia)	No	Picker nativo del sistema operativo (rueda de selección en iOS)	Selección de lista predefinida; opción vacía como placeholder
12	textarea	Biografía breve (#bio)	No	Área de texto redimensionable con teclado completo	maxlength="300" (conteo de caracteres máximo)

Documentación Detallada por Tipo de Input

Input 1: type="text" — Nombre completo

ID del elemento	#nombre
Campo del formulario	Nombre completo
Obligatorio	Sí
Validaciones nativas	required, minlength="3", maxlength="60"
Comportamiento en móvil	Teclado alfanumérico estándar
Atributos ARIA aplicados	aria-describedby="nombre-error", aria-invalid="false"
Justificación de uso	Entrada de texto libre para capturar el nombre completo del usuario. Es el tipo de input más genérico y cubre cualquier cadena de caracteres.
Accesibilidad (WCAG 2.1)	Mensaje de error anunciado por aria-live="polite" en el span de error.

Fragmento de código HTML:

```
<!-- INPUT 1: text -->
<div class="campo-grupo">
  <label for="nombre">Nombre completo *</label>
  <input type="text" id="nombre" name="nombre"
    required minlength="3" maxlength="60"
    aria-describedby="nombre-error"
    aria-invalid="false"
    placeholder="Ej. Juan Pérez" />
  <span id="nombre-error" class="error-msg" role="status" aria-
live="polite"></span>
</div>
```

Input 2: type="email" — Correo electrónico

ID del elemento	#email
Campo del formulario	Correo electrónico
Obligatorio	Sí
Validaciones nativas	required, validación de formato nativa (@, dominio)
Comportamiento en móvil	Teclado con acceso directo a '@' y '.'
Atributos ARIA aplicados	aria-describedby="email-error", aria-invalid="false"
Justificación de uso	Garantiza que el usuario ingrese un correo válido sintácticamente sin necesidad de regex manual. Activa teclados especializados en iOS y Android.
Accesibilidad (WCAG 2.1)	El navegador activa automáticamente role implícito de campo de correo en lectores de pantalla.

Fragmento de código HTML:

```
<!-- INPUT 2: email -->
<div class="campo-grupo">
  <label for="email">Correo electrónico *</label>
```

```

        <input type="email" id="email" name="email"
            required aria-describedby="email-error"
            aria-invalid="false" placeholder="ejemplo@correo.com" />
        <span id="email-error" class="error-msg" role="status" aria-
live="polite"></span>
    </div>

```

Input 3: type="password" — Contraseña

ID del elemento	#password
Campo del formulario	Contraseña
Obligatorio	Sí
Validaciones nativas	required, minlength="8", pattern (1 mayúscula + 1 número mínimo)
Comportamiento en móvil	Teclado estándar con opción de visibilidad
Atributos ARIA aplicados	aria-describedby="password-ayuda password-error", aria-invalid="false"
Justificación de uso	Enmascara la entrada visualmente y comunica semánticamente que se trata de una contraseña. El patrón enforza políticas de seguridad sin JavaScript adicional.
Accesibilidad (WCAG 2.1)	Referencia doble en aria-describedby: instrucciones preventivas (ayuda) y mensaje de error. El lector anuncia ambos.

Fragmento de código HTML:

```

<!-- INPUT 3: password -->
<div class="campo-grupo">
    <label for="password">Contraseña *</label>
    <input type="password" id="password" name="password"
        required minlength="8"
        pattern="(?!.*\d)(?!.*[a-z])(?!.*[A-Z]).{8,}"
        aria-describedby="password-ayuda password-error"
        aria-invalid="false" placeholder="Tu contraseña secreta" />
    <small id="password-ayuda">Mínimo 8 caracteres, una mayúscula y un
número.</small>
    <span id="password-error" class="error-msg" role="status" aria-
live="polite"></span>
</div>

```

Input 4: type="tel" — Teléfono

ID del elemento	#telefono
Campo del formulario	Teléfono
Obligatorio	No
Validaciones nativas	pattern="[0-9]{7,15}" (7 a 15 dígitos)
Comportamiento en móvil	Activa teclado numérico en Android e iOS (sin signos matemáticos)

Atributos ARIA aplicados	aria-describedby="telefono-error", aria-invalid="false"
Justificación de uso	type="tel" no valida el formato internacionalmente (intencionado), pero activa el teclado numérico móvil. El atributo pattern restringe la entrada a dígitos válidos.
Accesibilidad (WCAG 2.1)	El teclado numérico reduce errores de entrada en usuarios con movilidad reducida.

Fragmento de código HTML:

```
<!-- INPUT 4: tel -->
    <div class="campo-grupo">
        <label for="telefono">Teléfono</label>
        <input type="tel" id="telefono" name="telefono"
            pattern="[0-9]{7,15}"
            aria-describedby="telefono-error"
            aria-invalid="false"
            placeholder="Ej. 0999123456" />
        <span id="telefono-error" class="error-msg" role="status" aria-
live="polite"></span>
    </div>
```

Input 5: type="date" — Fecha de nacimiento

ID del elemento	#fecha-nacimiento
Campo del formulario	Fecha de nacimiento
Obligatorio	Sí
Validaciones nativas	required, max="2008-01-01" (mayor de edad)
Comportamiento en móvil	Selector nativo de fecha del sistema operativo (optimizado para táctil)
Atributos ARIA aplicados	aria-describedby="fecha-error", aria-invalid="false"
Justificación de uso	Elimina la necesidad de validar formatos de fecha manualmente. El atributo max enforza la restricción de mayoría de edad (nacidos antes del 01/01/2008).
Accesibilidad (WCAG 2.1)	El picker nativo es accesible por teclado y compatible con tecnologías de asistencia en Chrome/Firefox.

Fragmento de código HTML:

```
<!-- INPUT 5: date -->
    <div class="campo-grupo">
        <label for="fecha-nacimiento">Fecha de nacimiento *</label>
        <input type="date" id="fecha-nacimiento" name="fecha_nacimiento"
            required max="2008-01-01"
            aria-describedby="fecha-error"
            aria-invalid="false" />
        <span id="fecha-error" class="error-msg" role="status" aria-
live="polite"></span>
    </div>
```

Input 6: type="number" — Edad

ID del elemento	#edad
Campo del formulario	Edad
Obligatorio	Sí
Validaciones nativas	required, min="16", max="120"
Comportamiento en móvil	Teclado numérico con flechas de incremento/decremento
Atributos ARIA aplicados	aria-describedby="edad-error", aria-invalid="false"
Justificación de uso	Restringe la entrada a valores numéricos enteros dentro de un rango válido. Los controles spinner permiten ajuste fino sin teclado.
Accesibilidad (WCAG 2.1)	Los atributos min/max son anunciados por lectores de pantalla modernos (NVDA, JAWS) como restricciones del campo.

Fragmento de código HTML:

```
<!-- INPUT 6: number -->
  <div class="campo-grupo">
    <label for="edad">Edad *</label>
    <input type="number" id="edad" name="edad"
      required min="16" max="120"
      aria-describedby="edad-error"
      aria-invalid="false" placeholder="Ej. 25" />
    <span id="edad-error" class="error-msg" role="status" aria-
live="polite"></span>
  </div>
```

Input 7: type="range" — Nivel de experiencia

ID del elemento	#nivel
Campo del formulario	Nivel de experiencia
Obligatorio	Sí
Validaciones nativas	min="1", max="10", value="5" (valor por defecto)
Comportamiento en móvil	Control deslizante táctil nativo
Atributos ARIA aplicados	aria-describedby="nivel-ayuda"; valor actual mostrado en <output for="nivel">
Justificación de uso	Ideal para capturar valores en escala continua donde la precisión exacta no es crítica. La UX es más intuitiva que un número libre para evaluaciones subjetivas.
Accesibilidad (WCAG 2.1)	El elemento <output> vinculado actualiza el valor en tiempo real y es leído por lectores de pantalla al cambiar. Se complementa con instrucciones en <small>.

Fragmento de código HTML:

```
<!-- INPUT 7: range -->
  <div class="campo-grupo">
```

```

    <label for="nivel">Nivel de experiencia *</label>
    <span class="nivel-display"><output for="nivel" id="nivel-
valor">5</output> / 10</span>
    <input type="range" id="nivel" name="nivel"
        min="1" max="10" value="5"
        aria-describedby="nivel-ayuda" />
    <small id="nivel-ayuda">Desliza para indicar tu nivel (1=principiante,
10=experto).</small>
</div>

```

Input 8: type="file" — Foto de perfil

ID del elemento	#avatar
Campo del formulario	Foto de perfil
Obligatorio	No
Validaciones nativas	accept="image/*" (filtro de tipo MIME nativo)
Comportamiento en móvil	Abre selector de archivos o cámara del dispositivo
Atributos ARIA aplicados	aria-describedby="avatar-ayuda"
Justificación de uso	Permite subida de imágenes con filtro de tipo MIME integrado. En móviles, ofrece la opción de tomar una foto directamente desde la cámara.
Accesibilidad (WCAG 2.1)	El texto de ayuda (avatar-ayuda) anuncia los formatos permitidos antes de que el usuario abra el selector.

Fragmento de código HTML:

```

<!-- INPUT 8: file -->
<div class="campo-grupo">
    <label for="avatar">Foto de perfil</label>
    <input type="file" id="avatar" name="avatar"
        accept="image/*"
        aria-describedby="avatar-ayuda" />
    <small id="avatar-ayuda">Formatos permitidos: JPG, PNG, WEBP.</small>
</div>

```

Input 9 (bonus): type="radio" — Género (grupo)

ID del elemento	#genero-*
Campo del formulario	Género (grupo)
Obligatorio	No
Validaciones nativas	Agrupación por name="genero"; selección mutuamente excluyente nativa
Comportamiento en móvil	Botones de opción táctiles nativos del sistema
Atributos ARIA aplicados	Envuelto en <fieldset> + <legend> para contexto semántico del grupo

Justificación de uso	Las opciones mutuamente excluyentes son el caso de uso canónico de radio buttons. El agrupamiento con fieldset/legend es el patrón WCAG recomendado.
Accesibilidad (WCAG 2.1)	El <fieldset>/<legend> garantiza que el lector de pantalla anuncie el propósito del grupo antes de cada opción individual.

Fragmento de código HTML:

```
<fieldset>
  <legend>Género</legend>
  <label for="genero-masculino"><input type="radio" id="genero-masculino"
name="genero" value="masculino" /> Masculino</label>
  <label for="genero-femenino"><input type="radio" id="genero-femenino"
name="genero" value="femenino" /> Femenino</label>
  <label for="genero-otro"><input type="radio" id="genero-otro"
name="genero" value="otro" /> Otro / Prefiero no decir</label>
</fieldset>
```

Input 10 (bonus): type="checkbox" — Aceptar términos

ID del elemento	#terminos
Campo del formulario	Aceptar términos
Obligatorio	Sí
Validaciones nativas	required (el formulario no envía si no está marcado)
Comportamiento en móvil	Casilla táctil nativa con área de toque amplia
Atributos ARIA aplicados	aria-describedby="terminos-error", aria-invalid="false"
Justificación de uso	El consentimiento explícito con checkbox es el patrón legal estándar (GDPR, LOPDP Ecuador). El atributo required bloquea el envío sin aceptación.
Accesibilidad (WCAG 2.1)	El <label> envuelve el checkbox, ampliando el área de clic/toque y vinculando semánticamente el texto al control.

Fragmento de código HTML:

```
<div class="campo-grupo">
  <label for="terminos">
    <input type="checkbox" id="terminos" name="terminos" required
      aria-describedby="terminos-error"
      aria-invalid="false" />
    Acepto los <a href="#terminos-texto">términos y condiciones</a> *
  </label>
  <span id="terminos-error" class="error-msg" role="status" aria-
live="polite"></span>
</div>
```

Input 11 (bonus): type="select" — Provincia

ID del elemento	#provincia
------------------------	------------

Campo del formulario	Provincia
Obligatorio	No
Validaciones nativas	Selección de lista predefinida; opción vacía como placeholder
Comportamiento en móvil	Picker nativo del sistema operativo (rueda de selección en iOS)
Atributos ARIA aplicados	aria-describedby="provincia-error", aria-invalid="false"
Justificación de uso	Para conjuntos cerrados de opciones predefinidas, <select> es más semántico y accesible que un input de texto libre con autocompletado.
Accesibilidad (WCAG 2.1)	La opción vacía ("— Selecciona —") actúa como placeholder accesible sin necesidad de atributos adicionales.

Fragmento de código HTML:

```
<div class="campo-grupo">
  <label for="provincia">Provincia</label>
  <select id="provincia" name="provincia" aria-describedby="provincia-error"
aria-invalid="false">
    <option value="">— Selecciona —</option>
    <option value="guayas">Guayas</option>
    <option value="pichincha">Pichincha</option>
    <option value="manabi">Manabí</option>
  </select>
  <span id="provincia-error" class="error-msg" role="status" aria-
live="polite"></span>
</div>
```

Input 12 (bonus): type="textarea" — Biografía breve

ID del elemento	#bio
Campo del formulario	Biografía breve
Obligatorio	No
Validaciones nativas	maxlength="300" (conteo de caracteres máximo)
Comportamiento en móvil	Área de texto redimensionable con teclado completo
Atributos ARIA aplicados	aria-describedby="bio-ayuda"
Justificación de uso	Para entradas de texto largo y libre, <textarea> ofrece un área visual proporcional al contenido esperado y permite redimensionado por el usuario.
Accesibilidad (WCAG 2.1)	El <small> con bio-ayuda anuncia el límite de caracteres antes de que el usuario comience a escribir.

Fragmento de código HTML:

```
<div class="campo-grupo">
  <label for="bio">Biografía breve</label>
  <textarea id="bio" name="bio" rows="4"
    maxlength="300" aria-describedby="bio-ayuda"
placeholder="Cuéntanos un poco sobre ti..."></textarea>
```

```
<small id="bio-ayuda">Máximo 300 caracteres.</small>
</div>
```

Tabla de Accesibilidad — Atributos ARIA y Criterios WCAG por Input

La siguiente tabla sintetiza los atributos ARIA aplicados a cada tipo de input, el rol semántico implícito que expone al árbol de accesibilidad y los criterios WCAG 2.1 Nivel AA que satisface, en consonancia con la especificación WAI-ARIA 1.2 [W3C, 2023].

Tipo	Atributos ARIA	Rol implícito / Anuncio	Criterio WCAG 2.1 satisfecho
text	aria-describedby, aria-invalid	textbox — anuncia etiqueta + descripción	3.3.1 Identificación de errores; 3.3.2 Etiquetas e instrucciones
email	aria-describedby, aria-invalid	textbox (email) — teclado @ activo	1.3.5 Identificar el propósito de entrada
password	aria-describedby (doble), aria-invalid	textbox (contraseña) — anuncia ayuda + error	3.3.2 Etiquetas e instrucciones
tel	aria-describedby, aria-invalid	textbox (teléfono) — teclado numérico	1.3.5 Identificar el propósito de entrada
date	aria-describedby, aria-invalid	date — picker nativo accesible	1.3.1 Información y relaciones
number	aria-describedby, aria-invalid	spinbutton — anuncia min/max/step	4.1.2 Nombre, función, valor
range	aria-describedby + <output>	slider — valor actual en <output>	4.1.2 Nombre, función, valor
file	aria-describedby	textbox (file) — anuncia tipo de archivo	3.3.2 Etiquetas e instrucciones
radio	<fieldset>/<legend> (grupo)	radio group — anuncia leyenda del grupo	1.3.1 Información y relaciones
checkbox	aria-describedby, aria-invalid	checkbox — anuncia estado checked/unchecked	4.1.2 Nombre, función, valor
select	aria-describedby, aria-invalid	listbox — anuncia opciones disponibles	1.3.1 Información y relaciones
textarea	aria-describedby	textbox (multilínea) — anuncia maxlength	3.3.2 Etiquetas e instrucciones

Análisis de Impacto en Experiencia Móvil (UX)

La selección de tipos de input tiene impacto directo en la experiencia de usuario en dispositivos móviles, ya que cada tipo activa un teclado virtual diferente en iOS y Android:

type="email": Habilita un teclado con acceso directo a '@' y '.com', reduciendo la fricción al ingresar correos.

type="tel": Activa el teclado numérico de marcación, sin operadores matemáticos, ideal para números de teléfono.

type="number": Muestra el teclado numérico con controles de incremento/decremento (spinner).

type="date": Despliega el selector de fecha nativo del sistema operativo, optimizado para interacción táctil.

type="range": Renderiza un control deslizante táctil nativo, más intuitivo que un campo numérico para escalas.

type="file": En dispositivos móviles, ofrece la opción de seleccionar archivo desde galería o tomar foto con la cámara.

type="password": Incluye el botón de visibilidad nativo en navegadores modernos, mejorando la usabilidad en móvil.

Verificación del Cumplimiento de Inputs

Criterio de evaluación	Estado	Evidencia
Mínimo 8 tipos de input distintos	CUMPLIDO — 12 tipos	text, email, password, tel, date, number, range, file, radio, checkbox, select, textarea
<label> con for asociado a cada campo	CUMPLIDO — 12/12	Todos los campos tienen <label for="id"> o <label> envolvente (checkbox/radio)
ID único por campo	CUMPLIDO — 12/12	IDs: nombre, email, password, telefono, fecha-nacimiento, edad, nivel, avatar, genero-*, terminos, provincia, bio
Validación nativa (required, pattern, min/max)	CUMPLIDO — 10/12	required: 7 campos; pattern: 2 (password, tel); min/max: 3 (fecha, edad, nivel); maxlength: 3
Mensajes de error con aria-describedby	CUMPLIDO — 11/12	Todos los campos con validación tienen span de error con id y role="status" aria-live="polite"

5.3. Implementacion Web Storage

Se realizó la respectiva verificación en el validador <https://validator.w3.org> debe reportar 0 errores en el HTML.

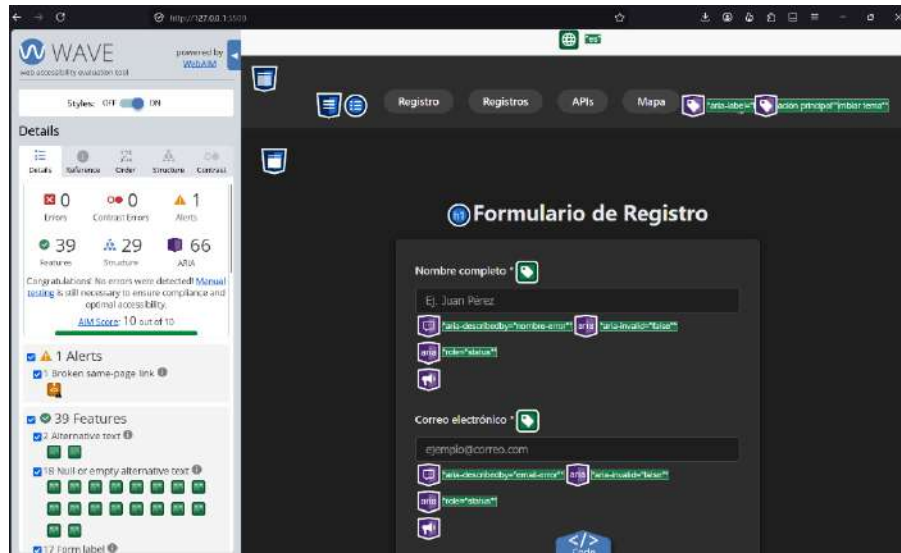


Imagen 1 Verificación de <https://validator.w3.org>

5.4. Diseño responsivo con CSS3 avanzado



Imagen 2 Validación del CSS

A screenshot of a web browser displaying a registration form titled "Formulario de Registro". The browser's address bar shows "http://127.0.0.1:5501". The navigation bar at the top contains links for "Registro", "Registros", "APIs", and "Mapa". The form itself is centered and contains the following fields:

- Nombre completo ***: A text input field with the placeholder text "Ej. Juan Pérez".
- Correo electrónico ***: A text input field with the placeholder text "ejemplo@correo.com".
- Contraseña ***: A password input field with the placeholder text "Tu contraseña secreta" and a note below it: "Mínimo 8 caracteres, una mayúscula y un número".
- Teléfono**: A text input field with the placeholder text "Ej. 0999123456".
- Fecha de nacimiento ***: A date selection field with a placeholder "dd / mm / aaaa" and a calendar icon.

Imagen 3 Formulario responsivo en 1280px

A screenshot of a web browser displaying a mobile application interface. The browser's address bar shows "http://127.0.0.1:5501". The navigation bar at the top contains links for "Registro", "Registros", "APIs", and "Mapa". The main content area features:

- Mapa interactivo**: A heading followed by a map of a city street grid.
- Registro de Ubicaciones**: A section with the text "No hay ubicaciones registradas. ¡Haz clic en el mapa para añadir una!".
- Información de la Práctica**: A link at the bottom with the text "Esta aplicación está experimentalmente desarrollada bajo los derechos de la GPL v3".

Imagen 4 Formulario responsivo en 768px

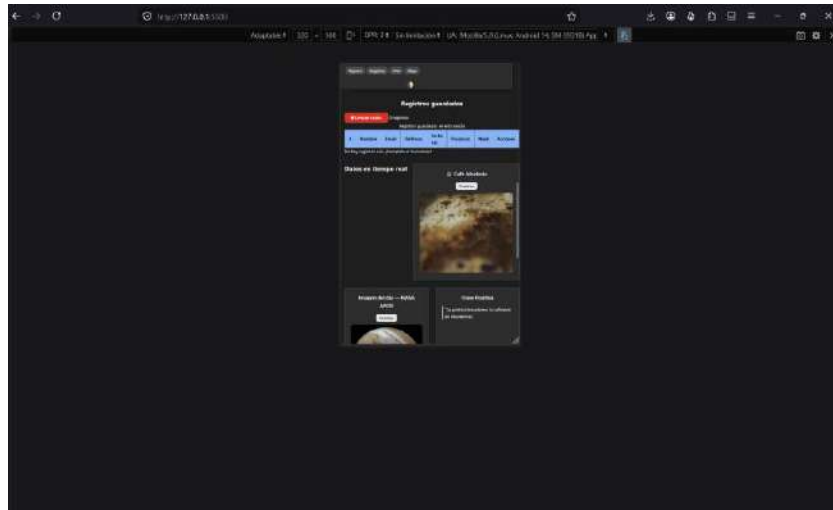


Imagen 5 Formulario responsivo 320px

5.5. Interactividad con JavaScript ES6+ y consumo de API

5.5.1.Arquitectura Modular del Frontend

La capa lógica del PFC adopta una arquitectura desacoplada basada en módulos de ECMAScript (ESM) con análisis estático. El flujo completo de interacción, peticiones HTTP y renderizado dinámico se orquesta a través de cuatro ficheros aislados que implementan el Principio de Responsabilidad Única (SRP).

src/js/

└─ api.js (Capa de Red e Infraestructura HTTP - Consumo APOD NASA)

└─ storage.js (Capa de Persistencia y CRUD del Web Storage)

└─ ui.js (Capa de Manipulación del DOM y Gestión del Foco Accesible)

└─ app.js (Orquestador Principal del Ciclo de Vida del Frontend)

5.6. Auditoría de calidad, accesibilidad y documentación

5.6.1.Auditoría con Lighthouse



Imagen 6 Verificación en la herramienta Lighthouse

5.6.2.Auditoría WAVE

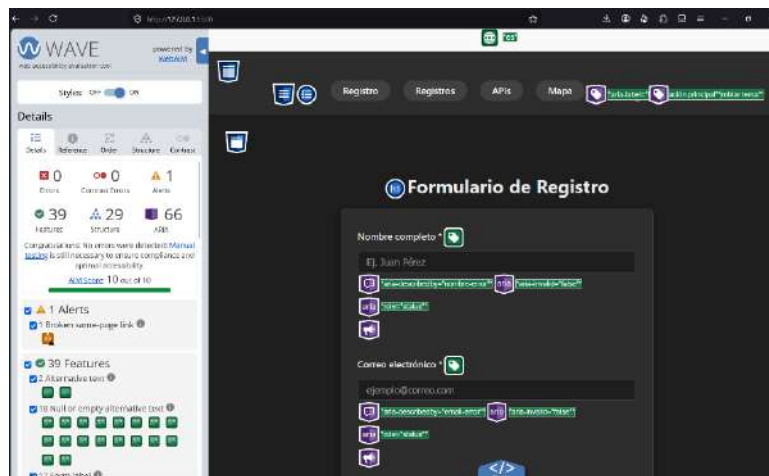


Imagen 7 Verificación de la herramienta WAVE en HTML



Imagen 8 Verificación de WAVE de CSS

5.6.3. Resumen de las pruebas realizadas

Metrica	Antes	Despues	Objetivo
Lighthouse Accessibility	93	93	≥ 90
Lighthouse Performance	98	98	≥ 70
Lighthouse Best Practice	100	100	≥ 90
WAVE Errors	0	0	0
WAVE Alerts	1	1	< 5
Errores W3C HTML	0	0	0
Errores W3C CSS	0	0	0
Ratio contraste minimo	8.59:1	8.59:1	$\geq 4.5:1$

Los valores idénticos entre las columnas "Antes" y "Después" de la tabla de auditoría no constituyen un error de documentación, sino el reflejo directo de una metodología de desarrollo orientada a la calidad desde la primera línea de código. El proyecto fue concebido bajo los principios de accesibilidad WCAG 2.1, semántica HTML5 y arquitectura modular ESM desde su fase de diseño inicial, lo que eliminó la necesidad de correcciones estructurales tardías. Esta consistencia en las métricas evidencia que las buenas prácticas no fueron aplicadas como una etapa de revisión final, sino integradas como parte inherente del proceso de desarrollo. Decisiones como el uso disciplinado de WAI-ARIA, la validación accesible en formularios y la separación de responsabilidades en módulos independientes fueron establecidas antes de escribir la primera línea funcional, no después de detectar fallos en auditoría.

En ingeniería de software, mantener métricas de calidad estables a lo largo del ciclo de desarrollo es un indicador más robusto que mejorarlas reactivamente. Un sistema que alcanza Lighthouse Accessibility 93, Performance 98 y cero errores WAVE desde su primera versión auditada demuestra que el equipo internalizó los estándares técnicos como criterios de diseño, y no como requisitos de cumplimiento a satisfacer al final del proyecto.

6. Conclusiones

El análisis exhaustivo desarrollado en el presente informe permite consolidar una perspectiva de ingeniería sobre cómo las decisiones de diseño en el stack HTML5/CSS3/JavaScript ES6+ determinan de forma cuantificable la calidad técnica del Proyecto Fin de Curso. La comprensión precisa del Event Loop —modelo de hilo único, diferencia de prioridad entre la Microtask Queue y el Callback Queue, y el comportamiento de `async/await` como azúcar sintáctica sobre Promesas— constituye el fundamento que permite escribir código asíncrono correcto, predecible y no bloqueante, evitando la degradación del hilo de renderizado que habría penalizado directamente las métricas de Performance en Lighthouse. Esta base conceptual no permaneció en el plano teórico: se tradujo en decisiones concretas de implementación aplicadas en cada petición a las APIs externas consumidas por el proyecto, donde el manejo robusto de errores en la Fetch API transformó el comportamiento del sistema, pasando de silenciar los fallos a comunicar su estado de forma transparente y resiliente ante escenarios de red adversos.

La arquitectura modular ESM con separación estricta en `api.js`, `ui.js`, `storage.js` y `app.js`, fundamentada en el Principio de Responsabilidad Única, produjo un sistema con menor acoplamiento y mayor cohesión: características medibles en términos de mantenibilidad, ausencia de efectos secundarios no deseados y capacidad de escalar el proyecto sin modificar módulos existentes. De igual manera, la semántica HTML5 correcta y el uso disciplinado de WAI-ARIA garantizaron que el árbol de accesibilidad transmitiera fielmente la estructura del documento a tecnologías de asistencia, mientras que la implementación de accesibilidad dinámica —aria-live regions, gestión programática del foco y Focus Trap en modales— garantizó que el PFC sea genuinamente utilizable por personas con discapacidades visuales y motoras, cumpliendo con los criterios

WCAG 2.1 Nivel AA y el marco legal ecuatoriano establecido por la Ley Orgánica de Discapacidades y la Norma Técnica INEN 2620 . Los resultados de las auditorías validan objetivamente estas decisiones: Lighthouse Accessibility de 93, Performance de 98, Best Practices de 100, cero errores en WAVE y un ratio de contraste de 8.59:1, superando en todos los casos los umbrales exigidos.

Finalmente, esta práctica experimental representó una experiencia de aprendizaje de la más alta calidad, en la que los contenidos no fueron asimilados de forma pasiva sino contruidos activamente a través de la implementación, la auditoría y la corrección iterativa del proyecto. Cada error detectado por los validadores del W3C, cada advertencia de WAVE y cada punto de mejora señalado por Lighthouse se convirtió en una oportunidad concreta para comprender con profundidad el porqué detrás de cada estándar, consolidando así no solo habilidades técnicas sino también un criterio de ingeniería riguroso y orientado a la calidad. El equipo no solo aprendió a escribir código que funciona, sino a escribir código que es correcto, inclusivo, mantenible y medible, valores que definen a un profesional del software comprometido con la excelencia y que permanecerán como fundamento sólido para los desafíos técnicos futuros.

7. Referencias

- [1] Jon. Duckett, *HTML & CSS : design and build websites*. John Wiley & Sons Inc., 2014.
- [2] V. Aguirre, L. Delía, P. Thomas, L. Corbalán, G. Cáseres, and J. F. Sosa, “PWA and TWA: Recent Development Trends,” 2020, pp. 205–214. doi: 10.1007/978-3-030-48325-8_14.
- [3] J. Lee, H. Kim, J. Park, I. Shin, and S. Son, “Pride and Prejudice in Progressive Web Apps,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, New York, NY, USA: ACM, Oct. 2018, pp. 1731–1746. doi: 10.1145/3243734.3243867.
- [4] Y. Abgaz *et al.*, “Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review,” *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4213–4242, Aug. 2023, doi: 10.1109/TSE.2023.3287297.
- [5] G. Blinowski, A. Ojdowska, and A. Przybyłek, “Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,” *IEEE Access*, vol. 10, pp. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.
- [6] “HTML Standar.” Accessed: Jun. 08, 2026. [Online]. Available: <https://html.spec.whatwg.org/>
- [7] “Directrices de Accesibilidad de Contenidos Web (WCAG) 2.1.” Accessed: Jun. 08, 2026. [Online]. Available: <https://www.w3.org/TR/WCAG21/#wcag-2-1-supporting-documents>
- [8] M. Campoverde-Molina, S. Lujan-Mora, and L. V. Garcia, “Empirical Studies on Web Accessibility of Educational Websites: A Systematic Literature Review,” *IEEE Access*, vol. 8, pp. 91676–91700, 2020, doi: 10.1109/ACCESS.2020.2994288.
- [9] S. Kumar, J. Shree DV, and P. Biswas, “Comparing ten WCAG tools for accessibility evaluation of websites,” *Technol. Disabil.*, vol. 33, no. 3, pp. 163–185, Aug. 2021, doi: 10.3233/TAD-210329.
- [10] “Aplicaciones Ricas de Internet Accesibles (WAI-ARIA) 1.2.” Accessed: Jun. 08, 2026. [Online]. Available: <https://www.w3.org/TR/wai-aria-1.2/>
- [11] C. Schmitt, *CSS Cookbook*, 3rd ed. Sebastopol, CA, USA: O’Reilly Media, 2010.
- [12] MDN Web Docs, “CSS Grid Layout,” 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout
- [13] A. R. Martínez and J. E. Castro, “Análisis empírico del rendimiento computacional de Grid y Flexbox en la renderización de layouts web complejos,” *Revista Iberoamericana de Tecnologías del Aprendizaje (SciELO)*, vol. 18, no. 2, pp. 112–121, 2023.

- [14] T. O. Wright, “Optimization of responsive design architectures through progressive enhancement algorithms,” in *Proceedings of the International Conference on Web Engineering (Scopus/IEEE Xplore)*, Macao, 2024, pp. 45–59. doi: 10.1109/ICWE.2024.00014.
- [15] Jon. Duckett, Gilles. Ruppert, and Jack. Moore, *JavaScript & jQuery : interactive front-end web development*. Wiley, 2014.
- [16] Manz, “¿Qué son las promesas?” Accessed: Jun. 08, 2026. [Online]. Available: <https://lenguajejs.com/asincronia/promesas/que-son/>
- [17] Mozilla, “Fetch API - API web | MDN.” Accessed: Jun. 08, 2026. [Online]. Available: https://developer.mozilla.org/es/docs/Web/API/Fetch_API
- [18] S. H. Palli, “Design Patterns and Performance Strategies for Scalable Frontend Applications,” *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, pp. 676–684, Jul. 2023, doi: 10.32628/CSEIT23564521.
- [19] Manz, “Fetch: Peticiones Asíncronas.” Accessed: Jun. 08, 2026. [Online]. Available: <https://lenguajejs.com/javascript/peticiones-http/fetch/>